

Genetic Programming Hyper-Heuristics for the Job Shop Scheduling Problem with Interval Durations: Robustness-Aware Interpretable Rules that Generalize Across Instance Sizes

Abstract

Automated dispatching-rule design has not been explored for the job shop scheduling problem with interval durations (IJSP), where processing times are known only to lie within intervals and schedules are built by interval arithmetic. We present a genetic programming hyper-heuristic that evolves priority rules over an *interval-aware* terminal set, exposing both the worst-case values and the widths of the uncertain quantities, and deploys them in a single constructive pass. On the 70 interval Taillard instances, 30 evolutions average 18.99% (± 1.33) relative error and the best rule attains 17.71%, improving on the strongest constructive baseline, Giffler–Thompson/MWKR (29.5%), on every instance. Evolved on four 20×15 instances, the rules transfer zero-shot to all size classes and to a second benchmark, remain human-readable, and are insensitive to the choice of training instances. A reimplementa-tion of a published genetic algorithm, sharing the rule’s decoder, needs some 7.6×10^4 schedule constructions to match one pass of the evolved rule, and, seeded with the rule, it outperforms every other method compared. Ablations delimit what interval awareness contributes. Under the default expected-makespan objective it is unnecessary: removing the width terminals, or evolving on crisp midpoints, does equally well. Its value lies in robustness: an objective penalizing interval width yields narrower

predicted intervals and reduces the deviation of the executed makespan by about one fifth, a gain that requires the width terminals and persists under non-uniform and worst-case realizations and on asymmetric intervals, at the cost of a higher expected makespan.

Keywords: interval job shop scheduling, genetic programming, hyper-heuristics, dispatching rules, scheduling under uncertainty, robustness

1. Introduction

In real manufacturing environments, the exact duration of an operation is rarely known in advance (Allahverdi, 2022). The interval job shop scheduling problem (IJSP) models this uncertainty in a minimal, distribution-free way: each processing time is only assumed to lie within a known interval, and schedules are built and evaluated with interval arithmetic, so that the makespan of a solution is itself an interval and solutions are compared by an interval ranking. Population metaheuristics for the IJSP, among them genetic algorithms and artificial bee colony variants, achieve strong results at the cost of building and evaluating large numbers of complete solutions for every instance (Díaz et al., 2020, 2023b,a).

At the opposite end of the computational spectrum are *dispatching rules*: priority functions that build a schedule in a single constructive pass by repeatedly selecting the next operation to start. They combine speed, interpretability and ease of deployment. However, hand-crafted rules leave a large quality gap with respect to search-based methods, as the experiments reported later in this paper confirm.

For *deterministic* scheduling, evolving rules automatically with genetic programming (GP) has been shown to narrow this gap substantially (Branke

et al., 2016; Nguyen et al., 2017). For scheduling under interval uncertainty, however, the automated design of dispatching rules remains, to the best of our knowledge, unexplored. Moving automated rule design to interval durations raises questions that do not arise in the deterministic case: which attributes the evolution actually selects, when the quantities describing a candidate operation (its duration, its earliest start, the work remaining in its job) are intervals rather than numbers; whether the *magnitude* of the uncertainty carries information that is useful for dispatching, or whether it is the position of the intervals that drives the decision; and whether an objective that rewards a narrow makespan interval, rather than a low one, leads the evolution to schedules whose execution departs less from what was predicted. Answering these questions empirically requires exposing uncertainty to the rule as first-class attributes, which is the starting point of this work.

A last question concerns the place of such rules among the methods already available for the IJSP. Dispatching rules and per-instance metaheuristics are usually compared at their natural operating points, one constructive pass against a complete search, which ranks them but does not say for which computational budget each is preferable. Placing both on a common budget axis, within a single implementation, answers that question: up to which budget an evolved rule is the better use of computation, and what separates it from the best search methods beyond it. A further question, how interval makespans should be compared at all when they admit no natural total order, we do not reopen: we adopt the ranking that a systematic study of the alternatives found most robust (Díaz et al., 2020).

This paper makes the following contributions:

1. A GP hyper-heuristic that brings automated rule design to the IJSP: priority expressions over a terminal set in which the duration, earliest start and work remaining of a candidate operation appear both as worst-case values and as widths, with fitness computed by interval arithmetic. The rules it produces are compact enough to be simplified and read by hand (Sections 4 and 7.1).
2. An empirical study of the evolved rules on the 70 interval Taillard instances comprising (i) a comparison with deterministic constructive methods at equal computational cost, (ii) an evaluation of zero-shot generalization across instance sizes and to a second benchmark of classical instances, and (iii) an analysis of sensitivity to the choice and size of the training set, measured against the variation between independent evolution runs (Sections 6 and 7.7).
3. A comparison with per-instance search, both against the published results of the IJSP metaheuristics and at matched budget against our Python implementation of the genetic algorithm published for the IJSP, in which the evolved rule, its sampled variant and the genetic algorithm, with and without the rule as a seed, share decoder and evaluator and are evaluated across budgets measured in schedule constructions (Sections 6.4 and 6.5).
4. A delimitation of what interval awareness contributes, in two parts: ablations that remove the width terminals or evolve on the crisp midpoint instances with scalar fitness, under the default expected-makespan objective (Section 7.4); and an objective that penalizes interval width, which has no crisp counterpart, whose effect on the deviation of the executed makespan from its prediction, in its mean and in its tail, is examined as a function of the weight of the penalty, under non-uniform

and worst-case realizations, and on asymmetric intervals (Sections 7.5, 7.6 and 7.8).

The remainder of the paper is organized as follows. Section 2 reviews related work and Section 3 formalizes the IJSP. Section 4 presents the GP hyper-heuristic, from both the hyper-heuristic and the evolutionary viewpoints. Section 5 describes the benchmarks, the data split, the reference methods, the configuration study, the evaluation measures and the computing environment. Section 6 reports the results: evolution, the position of the evolved rules among constructive methods, generalization, their comparison with the published metaheuristics, and a comparison at matched budget with our implementation of a genetic algorithm. Section 7 analyzes them: the anatomy of the rules and a worked example, the role of the decoder, the ablations of interval awareness, the robust objective and executional robustness, and the sensitivity of the results to the training set and to asymmetric intervals, before stating the limitations. Section 8 concludes.

2. Related Work

2.1. Job shop scheduling under interval uncertainty

Interval durations offer a distribution-free uncertainty model requiring only bounds, in contrast to stochastic and fuzzy approaches (Palacios et al., 2015): they are the natural choice when historical data are insufficient to estimate distributions and a decision maker can only supply a minimum and a maximum duration. Scheduling with interval processing times has been studied across machine environments, from the single machine (Allahverdi et al., 2014) to the flexible job shop (Xu et al., 2023), with the flow shop and others surveyed by Allahverdi (2022); recent work extends the model

to richer variants such as assembly (Zheng et al., 2024) and energy-aware flexible (Afşar et al., 2025) shops. It has also been studied analytically, through measures of the problem uncertainty induced by the intervals and of the stability of optimal sequences (Sotskov et al., 2013), and through the execution of schedules built under uncertain durations (Sotskov et al., 2020), the question Section 7.6 examines for evolved rules. The risk carried by a fixed schedule has also been quantified directly: Calamita et al. (2025) estimate the value-at-risk and conditional value-at-risk of its makespan when only the range of each duration is known, replacing costly exact evaluation with machine-learning predictors built on features of the activity network.

For the job shop with interval durations and makespan minimization, which concerns us here, the literature is dominated by population-based metaheuristics. Lei’s population-based neighborhood search (Lei, 2011), which ranks interval makespans by a possibility degree, was an early reference point; it was subsequently outperformed by a genetic algorithm with a systematic study of interval rankings (Díaz et al., 2020) and by artificial bee colony variants (Díaz et al., 2023b,a), which report the best published results for this problem. Related objectives have also been addressed under interval uncertainty, notably tardiness minimization (Díaz et al., 2022). All of these are iterative search methods: they build and evaluate many complete solutions and refine them over many iterations for each instance solved. Fast constructive methods for the IJSP have received far less attention, and dispatching rules with interval-aware lexicographic comparisons are the point of comparison they are measured against, the role they play in this paper (Section 6.2). Constructive heuristics are also a standard source of initial solutions for population methods (Kazimipour et al., 2014), a use Section 6.5 examines for evolved rules.

2.2. Dispatching rules for job shop scheduling

Priority dispatching rules are among the oldest and most widely deployed scheduling heuristics. Classical rules score each ready operation by a single attribute, and we adopt their standard acronyms throughout this paper. *Shortest* and *longest processing time* (SPT, LPT) look at the duration of the operation itself. *Earliest starting time* (EST) prefers the operation that can begin soonest given the current state of its job and its machine. *Most work remaining* (MWKR) and *most operations remaining* (MOR) look instead at the job the operation belongs to, and favor the one with the largest amount of pending work, measured respectively in time units and in number of operations. The *critical ratio* (CR) weighs the work still to be done against the time available until a due date. Surveys catalog more than a hundred such variants (Panwalkar and Iskander, 1977; Haupt, 1989), and more recent ones compare those in current use across machine environments (Đurašević and Jakobović, 2018). Beyond single-attribute rules, the algorithm of Giffler and Thompson (1960), G&T in what follows, restricts each choice to a machine’s conflict set, guaranteeing *active* schedules and markedly improving plain rules when a priority criterion is used to break ties inside that set; we write G&T-SPT and G&T-MWKR for the resulting combinations. Stronger constructive procedures exist for the *deterministic* job shop, notably the shifting bottleneck heuristic (Adams et al., 1988), which repeatedly solves one-machine subproblems with heads and tails by means of Carlier’s algorithm (Carlier, 1982). These fall outside the scope of the present work: their machinery rests on exact one-machine optimization and critical-path arguments that do not extend directly to interval arithmetic, where no natural total order is available. Hand-crafted rules, however, have reached a performance plateau: no fixed combination of attributes performs

well across shop configurations and objectives (Haupt, 1989), which motivated their *automated* design.

2.3. Automated design of dispatching rules

Genetic programming (Koza, 1992; Poli et al., 2008) evolves a population of expressions, usually represented as trees, by applying selection and variation operators directly on their structure. GP hyper-heuristics apply this machinery to *priority expressions* over scheduling attributes, and constitute the standard approach to automated rule design for production scheduling (Branke et al., 2016; Nguyen et al., 2017). Evolved rules have been shown to outperform hand-crafted ones across job shop variants, with active research on surrogate fitness (Hildebrandt and Branke, 2015), feature selection (Mei et al., 2017) and multi-objective formulations (Nguyen et al., 2014). Beyond single rules, a further line combines several evolved rules into *ensembles* that jointly construct or vote on a solution (Gil-Gala et al., 2021), and recent work explores alternatives to evolutionary search for the same design task, namely systematic tree search over the space of expression trees (Gil-Gala et al., 2025b), reporting rules of comparable quality but smaller size and better readability. All of this work targets deterministic (or dynamic-stochastic) settings. Closest to our setting, Karunakaran et al. (2017) evolve rules for dynamic job shops with uncertain processing times, exposing summary statistics of observed deviations (an exponential moving average) to the rule; uncertainty there is realized stochastically during simulation, and rules are evaluated on sampled scenarios. A recent survey of learned optimization for the job shop (Xu et al., 2025), covering both GP and DRL constructive approaches, records no work on interval-valued processing times. To the best of our knowledge, evolving rules whose *inputs*

are interval attributes, and whose fitness is computed by interval arithmetic rather than by sampling realizations, has not been explored before.

Finally, deep reinforcement learning provides an alternative family of learned constructive methods for the JSP (Zhang et al., 2020; Park et al., 2021); GP rules and DRL policies are increasingly studied as competing learning paradigms for the same construction task, and controlled comparisons between them under common protocols have been identified as a gap in the recent literature (Xu et al., 2025). Both families occupy the same computational class at deployment: a single constructive pass. Reinforcement learning has recently been applied to the IJSP itself: Han et al. (2026) learn which machine-order relations to fix before execution and leave those most sensitive to interval uncertainty to be resolved during execution by timed automata, reporting lower mean midpoint makespans than population-based neighborhood search and a genetic algorithm on 14 classical instances. The rules evolved here, by contrast, fix every sequencing decision before execution.

3. The Interval Job Shop Scheduling Problem

An IJSP instance consists of n jobs and m machines. Job j is a chain of m operations $o_{j1} \prec o_{j2} \prec \dots \prec o_{jm}$, each requiring a distinct machine $\mu(o_{jl})$ exclusively and non-preemptively. The duration of operation o is an interval $\mathbf{p}_o = [\underline{p}_o, \overline{p}_o]$ with $0 < \underline{p}_o \leq \overline{p}_o$: the only assumption is that the realized duration lies within these bounds. As in this definition, boldface denotes an interval-valued quantity throughout, and the underlined and overlined symbols its lower and upper bounds.

Schedule construction uses interval arithmetic. For intervals $\mathbf{a} = [\underline{a}, \overline{a}]$

and $\mathbf{b} = [\underline{b}, \bar{b}]$, the two operations required by makespan minimization are

$$\mathbf{a} + \mathbf{b} = [\underline{a} + \underline{b}, \bar{a} + \bar{b}], \quad (1)$$

$$\max(\mathbf{a}, \mathbf{b}) = [\max(\underline{a}, \underline{b}), \max(\bar{a}, \bar{b})], \quad (2)$$

both component-wise, the standard operations of interval arithmetic (Moore et al., 2009) as used in the IJSP literature (Díaz et al., 2020). A solution determines a task processing order π (a permutation with repetition of jobs, decoded left to right). Following the notation of Díaz et al. (2020) and Díaz et al. (2023a), let $\text{PM}_o(\pi)$ and PJ_o denote the predecessors of task o in its machine (according to π) and in its job. The *semi-active* schedule (Sprecher et al., 1995) starts each task at

$$\mathbf{s}_o(\pi) = \max(\mathbf{c}_{\text{PJ}_o}(\pi), \mathbf{c}_{\text{PM}_o(\pi)}(\pi)), \quad \mathbf{c}_o(\pi) = \mathbf{s}_o(\pi) + \mathbf{p}_o, \quad (3)$$

so starting and completion times of all tasks are themselves intervals. The problem can be stated as

$$\min_R \mathbf{C}_{\max} \quad (4)$$

$$\text{s.t. } \mathbf{C}_{\max} = \max_{1 \leq j \leq n} \{\mathbf{c}_{o_{jm}}\}, \quad (5)$$

$$\underline{s}_{o_{jl}} \geq \underline{c}_{o_{j,l-1}}, \quad \bar{s}_{o_{jl}} \geq \bar{c}_{o_{j,l-1}}, \quad 2 \leq l \leq m, \quad 1 \leq j \leq n, \quad (6)$$

$$\underline{s}_o \geq \underline{c}_{o'} \vee \underline{s}_{o'} \geq \underline{c}_o, \quad \bar{s}_o \geq \bar{c}_{o'} \vee \bar{s}_{o'} \geq \bar{c}_o, \quad \forall o \neq o' : \mu(o) = \mu(o'), \quad (7)$$

where Eq. (5) defines the interval makespan as the component-wise maximum of the job completion times, Eq. (6) enforces precedence within each job and Eq. (7) forbids overlaps on each machine, in both interval components (Díaz et al., 2020, 2023a). The minimum in (4) is taken with respect to an interval ranking R , since intervals admit no natural total order and any ranking embodies a modeling choice (Ishibuchi and Tanaka, 1990). We

rank schedules lexicographically, comparing upper bounds first and using the lower bound only to break ties:

$$\mathbf{a} \preceq \mathbf{b} \iff \bar{a} < \bar{b} \vee (\bar{a} = \bar{b} \wedge \underline{a} \leq \underline{b}), \quad (8)$$

one of the standard rankings of the IJSP literature and the one found by Díaz et al. (2020) to yield the most robust schedules. All methods in this paper, the GP rules and every baseline, share one implementation of this decoder and evaluator, so reported differences cannot stem from evaluator discrepancies.

Performance is reported as the relative error of the expected makespan with respect to a crisp reference bound LB of each instance,

$$\text{RE} = \frac{E[\mathbf{C}_{\max}] - \text{LB}}{\text{LB}} \times 100, \quad E[\mathbf{C}_{\max}] = \frac{\underline{C}_{\max} + \overline{C}_{\max}}{2}, \quad (9)$$

where $E[\mathbf{C}_{\max}]$ is the midpoint of the interval makespan, which the IJSP literature calls its expected makespan because it is the expectation under a uniform distribution on $[\underline{C}_{\max}, \overline{C}_{\max}]$; it serves here only as a point summary of the interval, and no distributional assumption enters the problem. LB is a bound of the underlying deterministic instance: Taillard’s bounds (Taillard, 1993) for the Taillard-derived instances, the best known ones (Díaz et al., 2023b) for the classical set. It serves as an indicative common reference, not as a bound for the interval problem, for which none is available.

A schedule is eventually *executed* under one realization of the durations, and a second quantity of interest is how well its a-priori interval predicts that execution. Measuring the gap between a predicted schedule and its execution is the classical notion of schedule robustness (Leon et al., 1994), here specialized to interval predictions. Following Díaz et al. (2020), the $\bar{\epsilon}$ robustness of a schedule with a fixed processing order is the average relative



Figure 1: An individual is a priority expression tree; leaves are interval-aware attributes of the candidate operation (here the illustrative rule $SLACK/PT + EST - WKR \cdot PTW$).

deviation of its executed makespan from the predicted expected value, over K realizations of the durations:

$$\bar{\varepsilon} = \frac{1}{K} \sum_{k=1}^K \frac{|C_{\max}^{\text{ex},k} - E[\mathbf{C}_{\max}]|}{E[\mathbf{C}_{\max}]}, \quad (10)$$

where $C_{\max}^{\text{ex},k}$ is the crisp makespan obtained by executing that same order under the durations of realization k . A low $\bar{\varepsilon}$ therefore means that the executed makespan stays close to its a-priori expectation.

4. GP Hyper-Heuristic for the IJSP

We describe the method from two complementary viewpoints: as a *hyper-heuristic*, that is, what an individual is and how it turns an IJSP instance into a schedule (Section 4.1); and as an *evolutionary algorithm*, that is, how the population of rules is initialized, varied and selected (Section 4.2).

4.1. Hyper-heuristic view: from expression trees to schedules

An individual is a priority expression tree (Figure 1). Leaves are drawn from the interval-aware terminal set of Table 1, and internal nodes from the function set

$$\mathcal{F} = \{ +, -, \times, \div_p, \min, \max, \text{neg} \}, \quad (11)$$

Table 1: Interval-aware terminal set, with exact definitions. For an eligible operation o of job j on machine $\mu(o)$: $\mathbf{p}_o = [\underline{p}_o, \bar{p}_o]$ is its duration interval; $\mathbf{C}_j$ and $\mathbf{C}_{\mu(o)}$ are the interval completion times of the last scheduled operation of job j and of machine $\mu(o)$; $\mathcal{R}_o$ is the set of operations of j after o ; $\mathcal{E}$ is the current eligible set. Terminals are evaluated on *raw* (unnormalized) values.

Terminal	Definition	Role
PT	$\bar{p}_o$	worst-case processing time
PTW	$\bar{p}_o - \underline{p}_o$	duration uncertainty
EST	$\bar{e}_o = \max(\bar{C}_j, \bar{C}_{\mu(o)})$	worst-case earliest start
ESTW	$\bar{e}_o - \underline{e}_o$, with $\underline{e}_o = \max(\underline{C}_j, \underline{C}_{\mu(o)})$	start uncertainty
WKR	$\sum_{q \in \mathcal{R}_o} \bar{p}_q$	worst-case work remaining
WKRW	$\sum_{q \in \mathcal{R}_o} (\bar{p}_q - \underline{p}_q)$	remaining-work uncertainty
NOR	$ \mathcal{R}_o $	operations remaining
SLACK	$\bar{e}_o - \min_{o' \in \mathcal{E}} \bar{e}_{o'}$	start slack within eligible set
ONE	1	constant

where neg is the unary sign change and $\div_p$ denotes protected division, defined as $a \div_p b = a/b$ if $|b| > 10^{-9}$ and a otherwise. The rule is deployed inside a semi-active schedule generation scheme (SGS): starting from the empty schedule, at each of the nm decision points the set $\mathcal{E}$ of eligible operations (those whose job predecessor is already scheduled) is formed, the tree is evaluated on the attributes of every $o \in \mathcal{E}$, and the operation with *lowest* priority value is appended at its earliest feasible start on its machine, Eq. (3).

Because the lowest value is dispatched, a criterion that seeks to *maximize* an attribute must be expressed through the unary neg operator. Classic rules are thus points of this search space: shortest processing time is the single-node tree PT , whereas most work remaining is $\text{neg}(WKR)$. We inject

both as seeded rules in the initial population.

Priority ties at dispatch time are broken by eligible set order (lowest job index first), which is deterministic; hence a rule maps each instance to exactly one schedule.

4.2. Evolutionary components

The initial population is built with the *ramped half-and-half* method (Koza, 1992; Poli et al., 2008): each individual draws its maximum depth uniformly from $\{2, 3, 4\}$ and is generated, with equal probability, by the *full* method, which expands every branch with function nodes down to that depth, or by the *grow* method, which may close any branch earlier by placing a terminal (probability 0.3 per node in our implementation). In both cases internal nodes are drawn uniformly from $\mathcal{F}$ and leaves uniformly from the terminal set. The last two members of the population are not generated at random: they are the two seeded rules of Section 4.1, inserted directly. Every individual is evaluated by the mean RE of its deterministic constructive pass over the four training instances.

Each generation then proceeds as follows. Parents are chosen by tournament, drawing k individuals uniformly without replacement from the current population and returning the fittest of them. With probability p_c two parents are recombined by *subtree crossover* (Koza, 1992; Poli et al., 2008): a node is drawn uniformly at random in each parent, leaves included, and the offspring is a copy of the first parent with the subtree rooted at its chosen node replaced by the one rooted at the second parent’s. Unlike the classical two-offspring formulation, we keep a single offspring per recombination. With the complementary probability $1 - p_c$ a single parent undergoes *subtree mutation* (Koza, 1992; Poli et al., 2008): a node is again drawn uniformly at

Algorithm 1 GP hyper-heuristic for the IJSP

```
1:  $P \leftarrow \text{ramped half-and-half}(\text{pop} - 2) \cup \{\text{SPT}, \text{MWKR}\}$ 
2:  $\text{evaluate}(P)$   $\triangleright$  mean RE of one constructive pass on the training
   instances
3: for  $g = 1$  to  $\text{gens}$  do
4:    $P' \leftarrow \text{elite}(P, e)$   $\triangleright$  elitism
5:   while  $|P'| < \text{pop}$  do
6:     if  $\text{rand}() < p_c$  then
7:        $p_1, p_2 \leftarrow \text{tournament}(P, k), \text{tournament}(P, k)$ 
8:        $\text{child} \leftarrow \text{subtreeCrossover}(p_1, p_2)$ 
9:     else
10:       $\text{child} \leftarrow \text{subtreeMutation}(\text{tournament}(P, k))$ 
11:    end if
12:     $P' \leftarrow P' \cup \{\text{child}\}$   $\triangleright$  fitness  $\infty$  if size  $> c$ 
13:  end while
14:   $P \leftarrow \text{evaluate}(P')$ 
15: end for
16: return best rule in  $P$ 
```

random, and the subtree rooted at it is discarded and replaced by a freshly generated random tree, built with the grow method and maximum depth 3 over the full primitive set. Offspring whose size exceeds a cap of c nodes receive infinite fitness and are thereby excluded from selection. The depth bounds above apply only to trees created from scratch, at initialization and inside mutation; during the run only the node count is bounded, so recombination may produce deeper expressions.

Replacement is fully generational, the offspring becoming the next pop-

Table 2: Evolution parameters: the configuration selected by irace (Section 5.4), used for every experiment reported in this paper unless a section states otherwise.

Population / generations	100 / 50
Initialization	ramped half-and-half (depths 2–4)
Seeded individuals	PT and $\text{neg}(WKR)$
Selection	tournament, size $k = 7$
Crossover / mutation prob.	$p_c = 0.77$ / 0.23 (subtree, depth ≤ 3)
Elitism	$e = 2$
Tree-size cap	$c = 30$ nodes (violations: fitness ∞)
Fitness	mean RE, deterministic rollout, TA11–TA14
Seeds	1–30 (RNG of evolution and environment)

ulation, except for the e best individuals of the current generation, which are copied into it unchanged. The process stops after a fixed number of generations and returns the best rule found. Algorithm 1 summarizes the generational loop and Table 2 lists the parameter values.

4.3. Randomized multi-start extension

A deterministic rule produces one schedule. For matched comparisons with sampling-based methods we wrap the evolved rule in GRASP-style randomization (Feo and Resende, 1995): with probability 0.1 the dispatched operation is drawn uniformly from the eligible set, otherwise the rule decides; sample 0 remains deterministic. The wrapper, which we call GP-best-of- N , reports the lexicographically best of its N schedules. This uniform perturbation of a single rule is the simplest member of a family of randomized rule-based builders; an alternative swaps among a *set* of evolved rules during construction (Gil-Gala et al., 2025a). GP-best-of- N is a device for comparison rather than a deployment mode: it places the evolved rule on

the same footing as sampling-based methods when methods are compared at a given number of schedule constructions and, as with GRASP and other multi-start constructive schemes, what improves with N is the dispersion of the sampled schedules rather than any individual decision. Its cost is reported in Sections 6.4 and 6.5.

5. Experimental Setup

5.1. Benchmark instances

We use two benchmarks, both taken from the IJSP literature (Díaz et al., 2020, 2023b,a): the 70 interval versions of Taillard’s instances (Taillard, 1993) (TA1–TA70, seven size classes from 15×15 to 50×20), and the 12 classical instances on which the published IJSP metaheuristics report results (FT10, FT20 (Fisher and Thompson, 1963), seven instances of the La family (Lawrence, 1984) and ABZ7–9 (Adams et al., 1988), from 10 to 20 jobs and 5 to 15 machines). Both were obtained from their crisp counterparts with the generation scheme introduced by Díaz et al. (2020): for every operation a value δ is drawn uniformly at random in $[0, 0.15 p]$, where p is its original duration, and that duration becomes the integer interval $[p - \delta, p + \delta]$, machine routes being unchanged. The interval is symmetric around p , so the crisp instance is exactly the midpoint scenario of its interval version. Since rules are evolved on Taillard instances only and applied to the classical set without any re-evolution, the latter doubles as a cross-family generalization test (Section 6.3). Both sets are released with this paper (see Data availability).

For the analysis of Section 7.8 the 70 Taillard instances were also regenerated with intervals skewed to the right. The crisp duration is recovered

from the midpoint of the symmetric version, which the scheme above preserves exactly, and the bounds are drawn as $p - \text{round}(pU[0, 0.05])$ and $p + \text{round}(pU[0, 0.25])$, so that a duration may fall slightly below its nominal value and considerably above it. Machine routes are unchanged and the crisp Taillard bounds still apply. Over the 37,250 operations the mean total width is 14.3% of p against 12.9% for the symmetric scheme: the two benchmarks differ above all in the shape of the intervals, and the asymmetric intervals are also wider on average, by 1.4 points of p .

5.2. Training protocol and data split

The 70 interval Taillard instances are split three ways. Rules are evolved on TA11–TA14 (20×15). TA15–TA20 form the validation set: they never contribute to fitness, but the design and configuration decisions of this paper, including the irace study of Section 5.4, were selected on their performance. The remaining 60 instances are unseen by every design and configuration decision and, together with the 12 classical instances, constitute the generalization test.

Thirty independent evolutions, differing only in their random seed, yield thirty rules. The one with the lowest mean RE on the validation set is the *featured rule*: the 60 held-out Taillard instances and the 12 classical ones play no part in this or in any other choice of the paper. Results are reported as the mean over the 30 rules, which no selection touches, beside the featured rule’s value, except where a section states that a single rule is examined. Section 7.7 repeats the whole protocol over five other training sets, of two, four and eight instances, and reports its effect.

5.3. Reference methods

The constructive baselines are the dispatching rules of Section 2 (SPT, LPT, EST, MWKR, MOR and CR) and the two Giffler–Thompson combinations, G&T-SPT and G&T-MWKR, each applied in one deterministic pass. Each ranks candidates by its own criterion under the lexicographic order of Eq. (8); Section 7.3 checks that this choice does not handicap them. A dispatcher that chooses uniformly among the eligible operations serves as a reference point.

The sampled variant of the evolved rule is GP-best-of- N (Section 4.3). The published IJSP metaheuristics are implemented in C++ and report results under their own stopping rules, which does not allow them to be compared with the evolved rule at a matched budget. For that comparison we implemented our own version of the genetic algorithm of Díaz et al. (2020) in Python, on the decoder and evaluator of Section 3, so that it and the evolved rule differ only in the method. A solution is a permutation with repetition of jobs, decoded by the semi-active scheme of Eq. (3) and compared by the ranking of Eq. (8). A population of 250 individuals is initialized at random; parents are chosen by tournaments of size 3 and recombined with probability 0.9 by a job-based order crossover, in which a random subset of jobs keeps its positions from the first parent and the remaining positions are filled in the order they take in the second; offspring are mutated with probability 0.2 by swapping two positions, and the two best individuals pass unchanged to the next generation. The population size is that of the published algorithm, and of four configurations tried at 2×10^5 constructions on the training instances it was the best. The algorithm is stopped after a given number of schedule constructions, which is the budget measure of Section 6.5.

Table 3: Hyperparameter space explored by irace (200 experiments; cost = validation-set RE of the evolved rule) and the winning configuration.

Parameter	Range / values	irace winner
Tournament size	{2, 3, 4, 5, 7}	7
Crossover prob. p_c	[0.5, 0.95]	0.77
Tree-size cap	{20, 30, 40, 60}	30
Elitism	{1, 2, 4}	2

5.4. Hyperparameter configuration with irace

Four parameters of the evolution were configured with irace (López-Ibáñez et al., 2016): tournament size, crossover probability, tree-size cap and elitism. Population size and number of generations were held fixed, so that every configuration is evaluated under the same evolution budget. The budget was 200 experiments, each one a full evolution on the training instances, scored by the resulting rule’s RE on the validation set. Table 3 lists the parameter space and the winning configuration (Table 2). Of the four elite configurations irace retains, three favor a high selection pressure (tournament size 7) and all four fall in a narrow crossover band, 0.72–0.77; the tree-size cap does not discriminate, the elites spanning caps of 20, 30 and 40.

5.5. Evaluation measures and statistical analysis

Quality is reported as the relative error RE of Section 3. The spread of a prediction is the relative width of the interval makespan, $W = (\overline{C}_{\max} - \underline{C}_{\max})/E[\mathbf{C}_{\max}] \times 100$, and executional robustness is measured by $\bar{\varepsilon}$ of Eq. (10) and by the absolute deviation $|\Delta|$, the same average taken before dividing by $E[\mathbf{C}_{\max}]$ and expressed in time units. Both are

averages over realizations; the tail of the same realizations is described by the conditional value-at-risk at level $\alpha = 0.95$, $\text{CVaR}_{0.95}$, the mean of the 5% largest values of a quantity, which is applied to the overrun $C_{\max}^{\text{ex}} - E[\mathbf{C}_{\max}]$ of the executed makespan beyond its prediction, in time units, and to the executed makespan itself, expressed as RE over the same bound LB. All of these are estimated by Monte Carlo over realizations of the durations, as described in Section 7.6. Lower is better on every measure.

Two kinds of comparison appear, with different experimental units. Methods are compared instance by instance, pairing their values on the same instances. Arms of evolutions, which differ in terminal set, fitness, training set or benchmark, are compared seed by seed, pairing the rules evolved with the same seed after averaging each rule over the instances. In both cases the test is the two-sided Wilcoxon signed-rank test with exact p -values, reported with the normal-approximation statistic z and, as effect size, the matched-pairs rank-biserial correlation $|r|$. Holm’s correction is applied within each family of related comparisons, and where it changes a verdict the text says so.

5.6. Computing environment

All experiments ran on a desktop computer with an AMD Ryzen 5 3400G processor (4 cores, 8 threads) and 32 GB of RAM under Windows 10 Pro. The hyper-heuristic, the decoder, the baselines and the genetic algorithm are implemented in Python 3.12 with NumPy 2.3, statistical tests use SciPy 1.17, and the configuration study used irace 4.4.3 (López-Ibáñez et al., 2016).

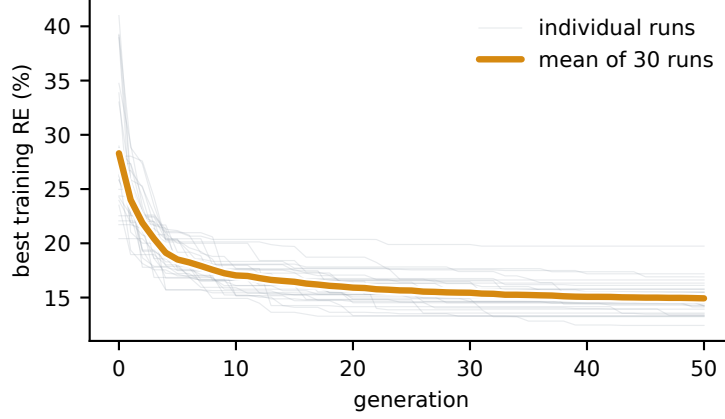


Figure 2: Best training fitness per generation over the 30 independent evolutions: each individual run and their mean.

6. Results

6.1. Evolution behavior and seed stability

Over 30 independent evolutions, training RE converges to 14.9 ± 1.5 (range 12.4–19.7), with most of the improvement realized by generation 25–30 (Figure 2). Figure 3 shows the featured rule, obtained with seed 1: 26 nodes over 12 leaves. Every terminal of the tree is non-negative by construction and processing times are at least one time unit, so both guards it contains resolve to the same branch at every decision. On the problem domain the tree is therefore exactly equivalent to the expression

$$SLACK^2 + 2PT - WKR - WKRW - 1. \quad (12)$$

We verified this identity at each of the 37,250 dispatching decisions the rule takes on the benchmark (1.29 million candidate evaluations).

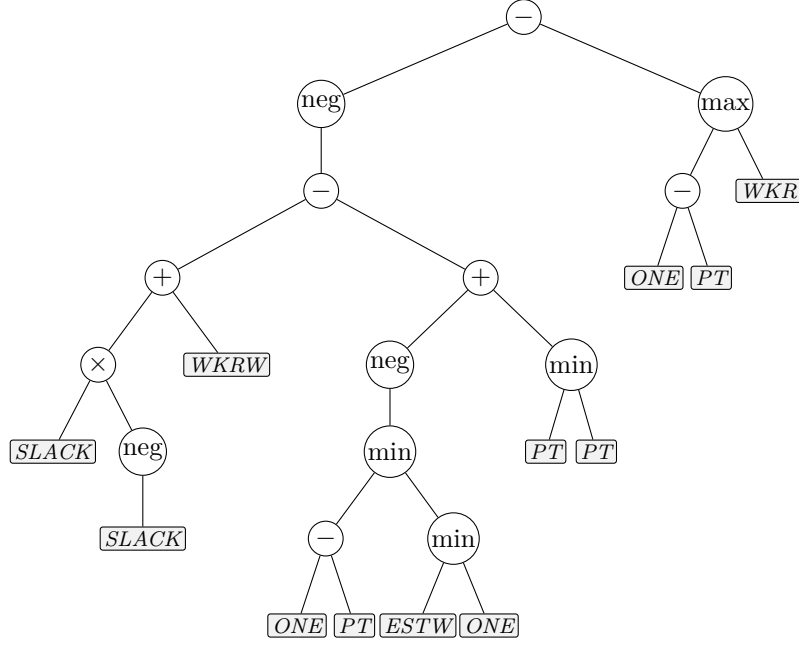


Figure 3: The featured rule (seed 1), which attains 17.71% RE over the 70 instances. Circles are functions and shaded boxes the interval-aware terminals of Table 1. The tree is shown exactly as evolved; Eq. (12) gives its simplified form.

6.2. Position among constructive methods

Table 4 reports every constructive baseline under the protocol of this paper, one deterministic pass per instance, together with the random dispatcher of Section 5.3. Since these rules are deterministic, the standard deviation is taken across instances and measures how uniform a method is over the benchmark rather than any run-to-run variability; the random dispatcher, the one stochastic entry, is averaged over 10 dispatches per instance. Times are wall-clock averages over the same 70 instances with 5 repetitions each; being implementation-bound, they are informative only in comparison between rows. The two GP entries are the mean over the 30 evolved rules and the best of them, the mean being what a single evolution should be

expected to yield.

Two observations are needed to read the table correctly. First, the plain rules that ignore machine availability (SPT, LPT and the critical ratio CR, the latter being a due-date rule applied here without due dates) perform *worse than random* in this setting. This is a property of the schedule generation scheme, not a defect of the criteria: because a solution is decoded as a permutation whose operations are appended to their machines, systematically dispatching by processing time alone produces highly unbalanced permutations. The same criterion embedded in the Giffler–Thompson conflict set, which restores classical dispatching semantics, behaves as expected (SPT: 627% as a plain rule versus 70.6% under G&T). Second, the criteria that do account for the state of the shop (EST, MOR, MWKR) are better than random, and the strongest baseline overall is G&T-MWKR at 29.5%, which is the reference we use throughout.

Against that strongest baseline, the evolved rules average 18.99 ± 1.33 RE over the 30 evolutions, and the best rule attains 17.71%.

The gap holds at the level of individual instances: the best evolved rule outperforms each of the eight deterministic baselines on every one of the 70 instances, and a paired Wilcoxon signed-rank test rejects equality against each of them at $p < 10^{-6}$ ($z = -7.3$); the conclusion is unchanged under Holm’s correction for the eight comparisons. The narrowest margin over the whole comparison is 3.9 points, against G&T-MWKR on a single 20×15 instance; against the best plain rule (EST) the smallest margin is 6.3 points. Per-instance figures are given in Table A.1 (appendix). This quality comes at no evaluation penalty over hand-crafted rules: evaluating the evolved expression tree over the eligible set costs what evaluating a hand-crafted criterion costs. Averaged over the 70 instances, a complete

Table 4: Constructive baselines on the 70 interval Taillard instances: mean RE (%), standard deviation across instances, and mean time in seconds of one constructive pass.

Method	RE (%)	sd	Time (s)
LPT	703.6	168.0	0.23
SPT	627.2	152.2	0.23
CR	625.7	140.6	0.26
<i>Random dispatcher</i>	<i>127.2</i>	<i>13.9</i>	<i>0.09</i>
G&T-SPT	70.6	10.0	0.29
MWKR	64.7	10.4	0.33
MOR	45.5	10.1	0.33
EST	42.3	10.6	0.28
G&T-MWKR	29.5	6.4	0.38
GP rule (mean of 30)	18.99	4.96	0.34
GP rule (best of 30)	17.71	5.23	0.34

constructive pass takes 0.34 s with the evolved rule against 0.33 s with MOR and 0.38 s with G&T-MWKR, a spread of 14% between the three; all of them reduce to one vectorized pass over the eligible operations per decision, so the choice of priority expression contributes little to the cost of a pass. Both cost and quality thus place the evolved rule with the constructive methods, while closing much of the quality gap that separated them from search-based methods; that comparison is the subject of Sections 6.4 and 6.5.

6.3. Generalization across sizes and instance families

Table 5 reports single-rollout RE per size class over all 70 instances, as mean $\pm$ standard deviation across the 30 evolved rules and for the single best rule. Evolved only on 20 $\times$ 15, the best rule attains its lowest RE on

Table 5: RE (%) per size class, single deterministic rollout, over all 70 Taillard interval instances: mean $\pm$ sd across the 30 independently evolved rules, and the best evolved rule.

	15 $\times$ 15	20 $\times$ 15	20 $\times$ 20	30 $\times$ 15	30 $\times$ 20	50 $\times$ 15	50 $\times$ 20	All
Mean of 30	18.1 $\pm$ 1.3	18.0 $\pm$ 1.5	20.9 $\pm$ 1.7	21.4 $\pm$ 1.7	25.7 $\pm$ 1.9	13.7 $\pm$ 1.5	15.1 $\pm$ 1.5	18.99 $\pm$ 1.33
Best rule	15.7	16.6	18.1	21.1	24.1	13.8	14.6	17.71

the unseen 50 $\times$ 15 (13.8%), and the same pattern holds on average (13.7 $\pm$ 1.5). Because all terminals are per-operation attributes with no size-bound quantities, transfer requires no retraining or rescaling.

The rules also transfer to the second benchmark, whose 12 classical instances differ from the Taillard ones in origin, size and structure. Table 6 gives the featured rule’s single-pass RE on each, together with GP-best-of- N at $N = 64$ and $N = 1024$ and the published results of the IJSP metaheuristics, which Section 6.4 discusses.

Transfer is a property of the method rather than of one rule: over the 30 evolved rules the deterministic pass averages $19.04 \pm 1.97\%$ RE on these instances, single rules ranging from 15.3% to 23.3%. Since the two benchmarks use different reference bounds (Section 3), RE values do not compare across them, but margins over baselines evaluated on the same instances do, and that margin is preserved: 2.40 times over MOR and 1.58 over G&T-MWKR on the classical set, against 2.40 and 1.55 on the benchmark the rules were evolved for.

6.4. Comparison with published metaheuristics

On the 12 classical instances the evolved rules can be compared with the published results of the IJSP metaheuristics on their own benchmark and metric (Table 6). Those methods belong to different algorithmic families, evolutionary computation and swarm intelligence: a genetic algorithm

Table 6: RE (%) on the 12 classical interval instances: our methods (deterministic single pass and GP-best-of- N) versus the published average results (30 runs per instance) of the IJSP metaheuristics GA (Díaz et al., 2020), ABC_{E3} , fEABC (Díaz et al., 2023b) and ESABC (Díaz et al., 2023a). LB is the best-known lower bound of the expected makespan.

Inst.	LB	Ours (zero-shot)			Published (avg of 30)			
		GP	GP-64	GP-1024	GA	ABC_{E3}	fEABC	ESABC
FT10	930	20.4	8.4	6.6	5.2	4.1	3.5	3.0
FT20	1165	13.2	12.1	10.8	4.4	1.7	1.8	1.8
La21	1046	15.0	14.4	9.0	5.0	5.0	4.2	4.0
La24	935	10.1	8.8	8.6	6.3	5.1	4.9	5.0
La25	977	11.8	11.6	7.6	5.1	3.9	3.4	2.7
La27	1235	25.0	13.7	11.3	10.2	4.7	4.6	4.1
La29	1152	13.4	14.2	11.0	14.2	8.6	7.4	7.0
La38	1196	15.0	13.3	9.4	9.2	6.9	6.1	5.8
La40	1222	13.3	12.8	9.9	8.7	4.2	4.6	4.1
ABZ7	656	19.7	14.3	11.7	12.5	7.3	6.6	6.7
ABZ8	645	24.7	22.1	17.7	18.5	12.1	11.1	10.9
ABZ9	661	33.4	25.3	22.1	18.0	13.1	11.6	11.2
Mean		17.9	14.2	11.3	9.8	6.4	5.8	5.5

evolving a population of permutations (Díaz et al., 2020) and three artificial bee colony variants (Díaz et al., 2023b,a), each of which runs a separate search on every instance.

The evolved rules are not competitive with the per-instance metaheuristics: the purpose-built ABC variants remain clearly ahead (5.5–6.4%), as expected of iterative search, and even wrapped as GP-1024 the featured rule reaches 11.3% against the genetic algorithm’s published 9.8%, better on 3 of the 12 instances (paired Wilcoxon $z = 1.96$, $p \approx 0.05$). What the

comparison locates is the constructive frontier: in a single pass the evolved rules average 19.04% on these instances, against 30.1% (G&T-MWKR) and 45.8% (MOR) for the deterministic baselines, and the best rule with a 1024-sample budget comes within 1.5 points of a population-based method that searches every instance separately.

The two families also differ in how they spend computation: GP-1024 evaluates 1024 mutually independent samples per instance, whereas the metaheuristics evolve a population of 250 individuals (Díaz et al., 2020, 2023b) through successive generations, each of which depends on the previous one. In wall-clock terms, a single constructive pass over these instances takes 0.03–0.13 s in our Python implementation, 0.07 s on average, against the 0.5–2.2 s reported for the genetic algorithm and the 1.9–6.8 s reported for fEABC on the same instances (Díaz et al., 2023b), both implemented in C++. GP-1024, however, requires 37–134 s of sequential Python time depending on instance size, and is therefore *not* faster than the published metaheuristics.

6.5. Comparison at matched budget with a reimplemented genetic algorithm

The comparisons of Section 6.4 set our Python implementation against published C++ results, each obtained under its authors’ own stopping rule, and do not show what either family attains at a given budget. For a fair comparison we use our Python implementation of the genetic algorithm of Díaz et al. (2020), described in Section 5.3. It is a faithful reference: on the 12 classical instances, averaged over three runs, it reaches 9.0% at 10^5 schedule constructions and 7.9% at 5×10^5 , ahead of the 9.8% published for the original. It remains short of the 5.5% of ESABC, and it levels off before reaching it, so the last few points that separate the evolved rules from

the best published methods are attributable to the neighborhood structures and local search that the ABC variants incorporate and a plain population method lacks, not to budget.

The comparison itself is run on the 70 Taillard instances. All methods share one simulator, an optimized implementation of the decoder of Section 3 that builds exactly the same schedules, which is why a pass of a rule takes 0.03 s here against the 0.34 s of Table 4. Five methods are compared: the featured rule in a single pass; its sampled variant GP-best-of- N , up to $N = 1024$; the genetic algorithm; the same algorithm started from the permutation the rule builds, which tests whether the rule can support a metaheuristic rather than compete with it; and uniformly sampled permutations, which is what a search with no structure attains. G&T-MWKR is added as a reference. Each stochastic method keeps its best-so-far solution, so one run yields its whole curve; the search methods run to 2^{17} constructions, with two runs per instance for the sampled rule and both genetic algorithms and one for random permutations.

The budget is measured in two ways. The number of schedules constructed does not depend on the implementation; the time in seconds does, and it also charges each method for what a construction costs it. Dispatching with a rule evaluates nine attributes for every eligible operation at each of the nm decisions, while decoding a fixed permutation evaluates none, so one pass of the rule costs what 36 to 134 decodings of the genetic algorithm cost, depending on instance size. That ratio is structural and would survive a reimplementaion; the absolute times would not. Table 7 gives the error of each method at fixed budgets in both measures, and Figure 4 gives the complete curves.

Measured in schedules constructed, the genetic algorithm starts from the

Table 7: RE (%) over the 70 Taillard instances at matched budgets, measured in schedules constructed and in seconds per instance. A dash marks a budget the method does not reach; a single pass is complete after 0.03 s and keeps its value at any later time.

Method	Schedules constructed					Seconds per instance		
	1	32	1024	2^{14}	2^{17}	1	5	20
Evolved rule, one pass	17.71	—	—	—	—	17.71	17.71	17.71
G&T-MWKR, one pass	29.52	—	—	—	—	29.52	29.52	29.52
Evolved rule, best-of- N	17.71	15.72	13.71	—	—	15.57	14.61	—
Genetic algorithm	112.09	82.26	66.60	33.80	15.84	60.29	41.76	24.01
Genetic algorithm, seeded	17.71	17.71	17.71	15.64	13.35	17.61	16.02	14.58
Random permutations	114.69	83.21	70.67	64.83	60.84	70.08	65.79	63.28

level of random sampling, 112% RE, and needs about 7.6×10^4 constructions to match the single pass of the rule at 17.71%. It does not reach what the same rule attains with 1024 samples, 13.71%: at 2^{17} constructions, 128 times the sampled rule’s budget, it reaches 15.84%. Over this whole range, from one construction to a hundred thousand, the evolved rule is the better use of the budget. Its advantage is not that it optimizes better, but that it is effective at budgets too small for search-based optimizers to be so.

Measured in seconds, the conclusion is stronger, although each construction of the rule is the more expensive one. One pass of the rule takes 0.03 s; after 20 s per instance, some six hundred times as long, the genetic algorithm is still at 24.01%, above the single pass on 43 of the 70 instances ($z = -3.72$, $p < 0.001$, $|r| = 0.51$). The sampled rule leads the genetic algorithm at every time where both were measured: 15.57% against 60.29% at 1 s, and 14.61% against 41.76% at 5 s, lower on all 70 instances ($z = -7.27$, $|r| = 1.00$). In a C++ implementation both curves would shift towards shorter times, but

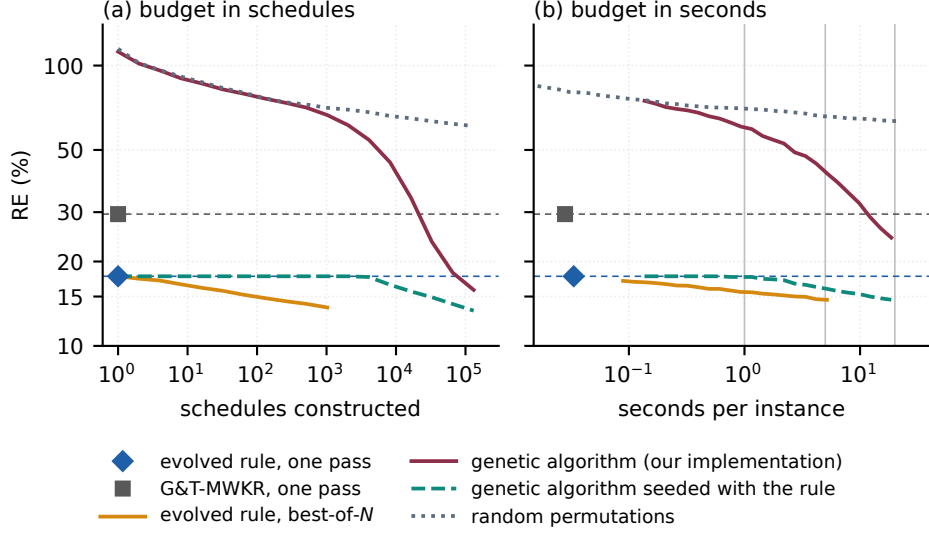


Figure 4: RE over the 70 Taillard instances against budget, measured in schedules constructed (a) and in seconds per instance (b). The two single-pass methods are points, drawn at their cost and extended as horizontal reference lines; the vertical lines in (b) mark the times of Table 7. No curve is drawn beyond the budget at which it was measured.

the ratio between the two costs, and with it the ordering, would remain.

The seeded genetic algorithm changes the interpretation. Started from the rule’s permutation instead of at random, it begins at 17.71% by construction and finishes at 13.35%, falling below the sampled rule at about 9.4×10^4 constructions; at 20 s it is at 14.58%, 9.43 points below the same algorithm started at random ($z = -7.00$, $|r| = 0.96$). It is the only method that finishes below the sampled rule, and it does so by starting from the rule. The natural place for an evolved dispatching rule alongside a per-instance metaheuristic is therefore not as its competitor but as its starting point, the role constructive heuristics commonly play in the initialization of popula-

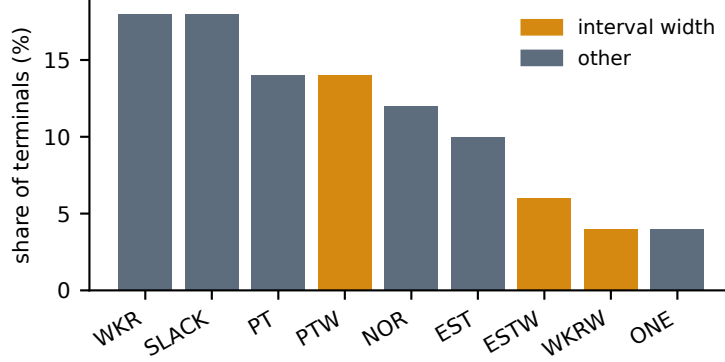


Figure 5: Terminal usage across the 30 evolved rules, as a share of all terminals. Interval-width terminals (amber) expose the magnitude of the uncertainty to the rule.

tion methods (Kazimipour et al., 2014), and the cheapest way to give such a method a good first incumbent.

7. Analysis

7.1. Rule anatomy and interpretability

The output of GP is a readable expression. Evolved rules are compact, with a mean tree size of 26 nodes (20–30) and mean depth 9, so a rule can be inspected and simplified by hand. Figure 5 reports how often each terminal is used across the 30 evolved rules. The most frequent are the classical scheduling attributes, slack (*SLACK*), work remaining (*WKR*) and earliest start (*EST*), together with worst-case processing time (*PT*), indicating that the evolution rediscovers known dispatching logic rather than exploiting benchmark artifacts. The *interval-width* terminals (*PTW*, *ESTW*, *WKRW*) account for $\approx 21\%$ of all terminal usage and appear in 27 of the 30 evolved rules. Whether that width information translates into better schedules is quantified in Sections 7.4–7.6.

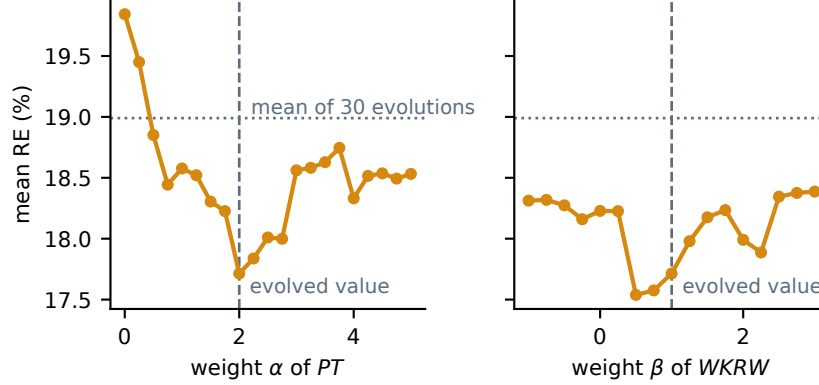


Figure 6: Sensitivity of the featured rule to the two weights of its simplified form, Eq. (12): mean RE over the 70 instances when each weight is varied and the other is held at its evolved value. Dashed lines mark the evolved weights, the dotted line the mean over the 30 evolved rules. $\beta = 0$ is the rule with its interval-width term switched off.

Since the lowest value is dispatched, the featured rule, Eq. (12), reads directly: $-WKR$ is most-work-remaining, $2PT$ is shortest-processing-time at double weight, $SLACK^2$ penalizes quadratically any operation that cannot start at the earliest available time, and the constant is immaterial to the ranking. The remaining term, $-WKRW$, is the interval-aware one: among jobs with comparable remaining work the rule advances those whose remaining work is *more uncertain*, resolving that uncertainty early rather than carrying it to the end of the schedule. The evolved rule is thus a weighted blend of MWKR and SPT, regularized by a quadratic earliest-start term and tilted by the uncertainty of the work ahead.

The simplified form of the featured rule, Eq. (12), also admits a direct sensitivity analysis: it weighs PT by $\alpha = 2$ and $WKRW$ by $\beta = 1$, and we swept each weight over the 70 instances holding the other at its evolved value (Figure 6). Neither curve is unimodal: over the sampled grid α has five

local minima and β three, and since each point is a deterministic evaluation over the 70 instances, that ruggedness belongs to the landscape rather than to sampling noise. The evolved $\alpha = 2$ is nonetheless the global minimum of its sweep; the evolved β is not, giving 17.71% against the 17.54% of $\beta = 0.5$. A post-evolution weight sweep is thus a cheap refinement for any simplified rule. The multimodality becomes relevant if the sweep is replaced by local descent: started at $\alpha = 4$, a descent converges to a secondary minimum 0.6 points above the global one. Setting $\beta = 0$ removes the width term from the deployed rule and costs 0.5 points of RE (18.23 against 17.71).

7.2. A worked example

A 3×3 instance makes the tilt introduced by $-WKRW$ visible. Instances were generated with the scheme of Section 5.1 and screened for one in which removing the width term, $\beta = 0$ in the notation of Section 7.1, changes exactly one dispatching decision and leaves the makespan worse for it; the behavior of the baselines played no part in the selection. The first such instance has three jobs of three operations: J_1 visits M_3, M_2, M_1 with durations $[28, 30]$, $[20, 26]$, $[24, 32]$; J_2 visits M_1, M_3, M_2 with $[26, 30]$, $[10, 12]$, $[16, 18]$; and J_3 visits M_3, M_1, M_2 with $[22, 26]$, $[14, 14]$, $[27, 31]$.

The two traces coincide in the first two decisions and diverge at the third, where M_3 has just been released by o_{11} and the eligible set is $\{o_{12}, o_{22}, o_{31}\}$. Table 8 gives the terminals and the resulting scores. All three candidates can start at the same time, so SLACK vanishes and the decision rests on the remaining terms. Without the width term the ranking puts o_{22} (5) ahead of o_{31} (6): the short operation of the job with little work remaining ranks first because of the $2PT$ term. The width term contributes $-WKRW$, which is -4 for o_{31} against -2 for o_{22} , and that reverses a one-point gap. The rule

Table 8: Terminals and priority scores at the third dispatching decision of the worked example.

Eligible	Duration	WKR	WKRW	SLACK	Eq. (12)	$\beta = 0$
o_{12}	[20, 26]	32	8	0	11	19
o_{22}	[10, 12]	18	2	0	3	<u>5</u>
o_{31}	[22, 26]	45	4	0	<u>2</u>	6

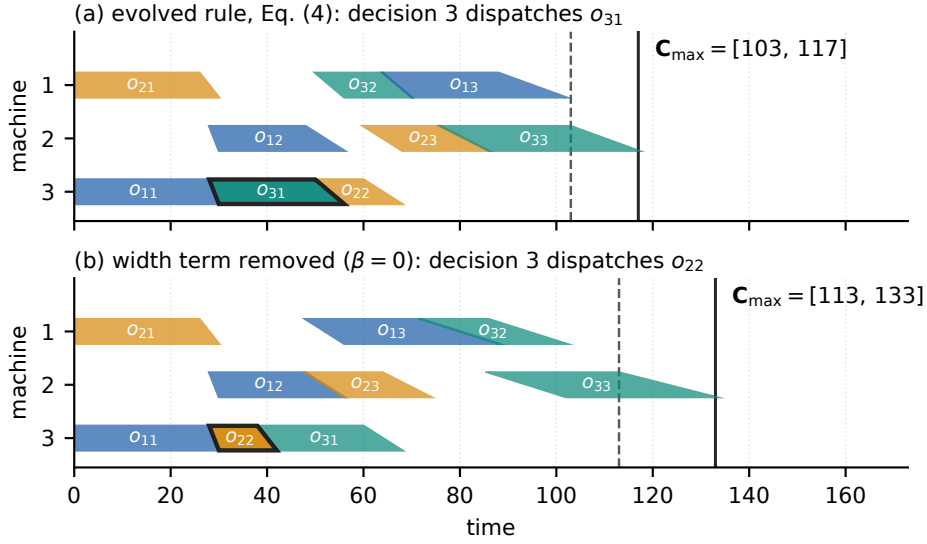


Figure 7: The worked example scheduled by the evolved rule and by the same rule without the width term. The outlined operation is the one that separates the two traces.

therefore dispatches o_{31} , the first operation of the job whose remaining work is the most uncertain, [41, 45] against [16, 18]. Of the two classical baselines, SPT dispatches o_{22} and MWKR dispatches o_{31} .

Figure 7 shows what follows. Dispatching J_3 early schedules its remaining operations while the machines are still available, and the makespan is $C_{\max} = [103, 117]$; delaying it by that single decision postpones the same

operations to the end of the schedule, and the makespan becomes [113, 133], worse at both endpoints. SPT and MWKR yield [147, 170] and [143, 161]: MWKR agrees with the evolved rule at this decision and diverges from it at a later one.

The example illustrates the mechanism, not its magnitude. Over the first 800 instances generated this way, removing the width term leaves the trace unchanged in 668; of the 132 traces that do change, 93 end at the same makespan, 21 end better with the term and 18 worse. At this scale the width terminals rarely affect a decision, and when they do their effect is barely distinguishable from chance, which is why their value is established over the 70-instance benchmark in Sections 7.4 and 7.6 rather than here.

7.3. Interval handling and the decoder

Two aspects of the comparison with the constructive baselines are examined here: how each baseline reduces an interval to a comparable quantity, and how much of the margin of the evolved rules belongs to the priority rule rather than to the schedule generation scheme.

The baselines rank candidates by the lexicographic order of Eq. (8) (Section 5.3). To check that this choice does not handicap them, each interval-valued rule was re-run ranking by a single number drawn from the same interval: its lower endpoint, its midpoint or its upper endpoint. Over the 70 instances SPT spans 633.1 to 642.5 across those three conventions, LPT 703.1 to 704.4, MWKR 62.9 to 64.6 and EST 42.7 to 45.1. The lexicographic values of Table 4 are better than all three for SPT and EST, and within 1.9 points of the best for LPT and MWKR. For EST and MWKR, the rules that account for the state of the shop, no convention moves the result by more than 2.4 points, and for SPT and LPT, whose levels exceed 600, by at

Table 9: RE (%) over the 70 instances for the two decoders and the two priority rules, one deterministic pass. The GP entries are the mean over the 30 evolved rules.

Priority rule	Semi-active	Giffler–Thompson
MWKR	64.7	29.5
Evolved, mean of 30	18.99 $\pm$ 1.33	23.18 $\pm$ 7.47
Evolved, featured	17.71	18.15

most 9.4; the best plain rule and the evolved one are 23.3 points apart. How the intervals are collapsed is therefore not what the comparison measures.

The decoder is a more substantial question, since the strongest baseline, G&T-MWKR, does not use the same one: it restricts each decision to Giffler and Thompson’s conflict set and so produces active schedules, whereas the evolved rules choose from the whole eligible set and produce semi-active ones. Running the evolved rules as the tie-break *inside* the conflict set separates the two effects. Table 9 gives the four combinations. Holding the decoder fixed, the evolved rules outperform MWKR under either one: 18.99 against 64.7 in the semi-active decoder and 23.18 against 29.5 in G&T’s. Holding the rule fixed, each prefers a different decoder: the evolved rules lose 4.2 points when moved into the conflict set, while MWKR gains 35.2 by entering it. Each method in Table 4 is thus reported with the decoder that suits it, and the margin survives the other choice.

Two details of the table deserve mention. The featured rule is the one case where the decoder barely matters, 17.71 against 18.15, a difference of 0.44 points that a paired test over the 70 instances does not separate from zero ($p = 0.32$); the loss is concentrated in the rest of the arm, whose standard deviation grows from 1.33 to 7.47 because a few rules deteriorate markedly once their candidate set is restricted. This is what should be

expected of rules evolved against a particular generation scheme: the conflict set removes candidates their fitness was never measured on. And SPT, which is the worst entry of Table 4 at 627.2, reaches 70.6 inside the conflict set, as G&T-SPT, which is the same phenomenon seen from the other side.

7.4. *The value of interval awareness: ablations*

The interval-width terminals are the distinctive ingredient of this setting, so we isolate their contribution by ablation: we repeat the 30 evolutions with and without the width terminals (*PTW*, *ESTW*, *WKRW*) in the terminal set, everything else held fixed, and compare the resulting rules seed by seed (Section 5.5); Holm’s correction over the tests of Table 10 changes none of their verdicts. The irace configuration of Section 5.4 was selected on the full terminal set and is used for both arms: holding it fixed is what makes the comparison controlled, at the cost that the ablated arm runs with hyperparameters tuned for a larger primitive set than its own. Table 10 collects the outcome.

Under the makespan objective, the width terminals do not improve the quantity being optimized. Removing the three of them leaves the expected makespan unchanged: 18.66 ± 1.08 mean RE over the 70 instances without them versus 18.99 ± 1.33 with them, a difference that is not significant ($z = -0.83$) and whose sign favors the smaller terminal set. They do, however, affect the schedules: the relative width of the predicted makespan interval is significantly narrower when the rule can read widths (12.41 ± 0.30 versus 12.62 ± 0.23 ; paired Wilcoxon $z = 3.40$, $p < 0.001$, $|r| = 0.71$), even though nothing in this objective rewards narrowness. Their effect under this objective is thus a slightly tighter interval at no cost in expected makespan.

Interval information enters the evolution at two points: through the

Table 10: Ablations of the interval machinery (30 independent evolutions per arm; paired rank tests between arms): the terminal ablation under two fitness objectives, and the crisp-midpoint control, evolved with scalar fitness and no interval information, tested against the interval-trained arm above it. *Width* is the relative width of the predicted makespan interval.

Fitness objective	Terminal set	RE (%)	Width (%)
Makespan $E[\mathbf{C}_{\max}]$	full	18.99 ± 1.33	12.41 ± 0.30
	without widths	18.66 ± 1.08	12.62 ± 0.23
	<i>test</i>	<i>n.s.</i> ($z = -0.83$)	$z = 3.40, p < 0.001$
Midpoint control	without widths	18.63 ± 0.99	12.55 ± 0.17
	<i>test</i>	<i>n.s.</i> ($z = -0.13$)	<i>n.s.</i> ($z = -1.29$)
Robust, Eq. (13)	full	19.62 ± 1.52	12.04 ± 0.75
	without widths	18.42 ± 0.87	12.47 ± 0.20
	<i>test</i>	$z = -3.75, p < 0.001$	$z = 3.22, p < 0.001$

terminal set, which fixes what a rule can read, and through the fitness, which scores every candidate schedule by interval arithmetic. The ablation above removes the first but keeps the second: rules evolved without width terminals are still selected by the interval-evaluated expected makespan.

To remove the second as well, we evolve 30 further rules on the crisp counterparts of the training instances, exactly their midpoint scenarios (Section 5.1), with the same seeds, configuration and reduced terminal set: these evolutions never handle interval data, and their fitness is the plain scalar makespan. Deployed unchanged on the interval benchmark, they are indistinguishable from the arm evolved with interval fitness, 18.63 ± 0.99 against 18.66 ± 1.08 RE ($z = -0.13$, *n.s.*), with predicted widths equally close (12.55 ± 0.17 against 12.62 ± 0.23 , $z = -1.29$, *n.s.*), although the two

fitness functions are not identical, since $\max$ does not commute with taking midpoints. For expected makespan, then, neither component is needed: rules evolved with no interval information at all match the interval-trained arm. The objective of the next section penalizes the width of the makespan interval and cannot be replicated on a crisp instance, where that width is identically zero.

7.5. The robust objective: trading expected makespan for predictability

The makespan fitness gives the evolution no incentive to trade expected makespan for predictability. To test whether it can make that trade when rewarded for it, we repeat the terminal ablation with a *robust* fitness that explicitly rewards predictability, minimizing

$$f_\lambda = \overline{C}_{\max} + \lambda (\overline{C}_{\max} - \underline{C}_{\max}), \quad (13)$$

that is, a low worst case *and* a narrow interval, with the weight λ setting the balance between the two; we take $\lambda = 1$ here. Under this objective the separation widens. Rules evolved with the width terminals produce schedules whose makespan interval is significantly narrower than those evolved without them (12.04 ± 0.75 versus 12.47 ± 0.20 relative width; paired Wilcoxon $z = 3.22$, $p < 0.001$, $|r| = 0.67$), at the cost of a higher expected makespan (19.62 versus 18.42 RE), which is the trade-off the robust objective is designed to make.

The weight λ of Eq. (13) governs that trade-off, and sweeping it traces the frontier between quality and predictability (Figure 8). Both quantities move monotonically with λ : as the weight grows from 0.5 to 4, the relative width of the makespan interval falls from 12.18 ± 0.50 to 10.54 ± 1.26 while the expected makespan degrades from 19.20 ± 1.52 to 23.27 ± 3.80 RE. At

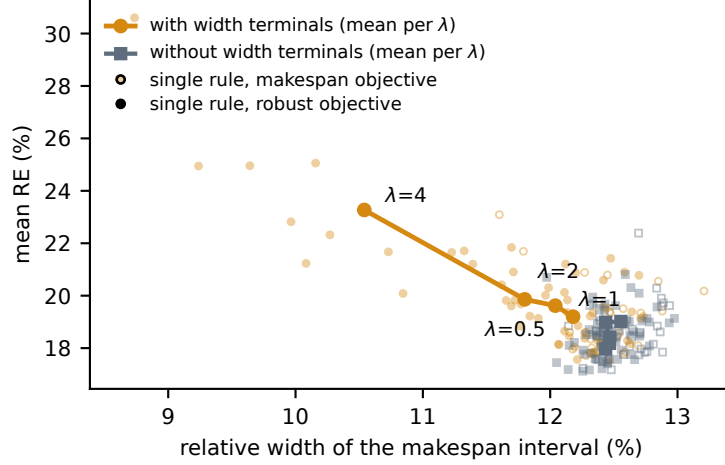


Figure 8: Quality–predictability plane over the 70 instances. Faint markers are individual evolved rules: amber with the interval-width terminals, grey without them; open markers for the makespan objective, filled for the robust one at any λ . Solid curves join the mean of each arm at each λ of the robust-fitness sweep, Eq. (13). With the width terminals, increasing λ trades expected makespan for narrower intervals; without them the five means, $\lambda = 0$ to 4, fall within 0.13 points of width and are left unlabeled.

the far end of the sweep the schedules are therefore narrower than any the ablated rules produced, at a cost of about four points of RE.

Read at the level of the arm, which is what a single evolution should be expected to deliver, the frontier has two regimes. Between $\lambda = 0.5$ and $\lambda = 2$ each point of narrower interval costs 1.7 points of RE, and the whole range costs 0.66 points; between $\lambda = 2$ and $\lambda = 4$ the rate rises to 2.7, the last 1.26 points of width costing 3.41 points of RE, and the spread between evolutions more than doubles, from 1.74 to 3.80, so the outcome of a single run becomes much less predictable. A weight between 1 and 2 is therefore the default we would recommend, and the largest weights are warranted only where predictability is worth considerably more than

expected makespan. Selecting a rule within an arm amplifies both sides of the trade-off: the narrowest rule of the $\lambda = 1$ arm, used in Section 7.6, reduces the executed deviation by 16.2% at a cost of 7.2 points of RE with respect to the featured rule. A practitioner concerned only with expected makespan can, by Section 7.4, use the smaller terminal set without loss.

Sweeping λ for the ablated arm (ten evolutions per value) shows that it has no such curve. Its width does not respond to the weight at all: 12.47 ± 0.16 at $\lambda = 0$, 12.43 ± 0.15 at 0.5, 12.47 ± 0.20 at 1, 12.44 ± 0.24 at 2 and 12.56 ± 0.17 at 4, a range of 0.13 points against the 1.64 the full arm covers, with the largest width at the largest weight, and with a spread several times smaller, and no individual rule departs from this band either. Penalizing width in the fitness therefore yields narrower intervals only when the rule has terminals through which to act on it: the objective and the representation are not two independent routes to the same end, the first works through the second.

The rule-level layer of Figure 8 shows the same separation rule by rule. Without the width terminals no rule goes below 12.05% under either objective; with them, the minimum is 11.60% under the makespan objective and 9.24% under the robust one. The terminals alone therefore suffice to leave the ablated band, but reaching 9.24% requires both them and an objective that rewards narrowness.

7.6. Robustness under executional uncertainty

We estimate the $\bar{\varepsilon}$ robustness of Eq. (10) by Monte Carlo, drawing $K=1000$ realizations of the durations uniformly on each operation’s interval and executing the fixed processing order on each of them. The sampling measures schedules that are already built. The processing order is fixed

Table 11: Predicted quality and executional robustness over the 70 instances: mean RE, relative width of the predicted makespan interval, and $\bar{\varepsilon} (\times 10^3)$ at nominal interval widths and widened by +20% and +40% (Monte Carlo, $K=1000$ realizations, common random numbers across methods). The last column is $|\Delta|$ at nominal widths. Each GP arm is represented by one rule: the makespan arms by their best RE, the robust arms by their narrowest predicted interval, which is what their fitness optimizes.

Method	RE (%)	Width (%)	$\bar{\varepsilon} (\times 10^3)$			$ \Delta $
			+0%	+20%	+40%	
GP, makespan	17.71	12.28	5.70	6.64	7.57	12.30
GP, makespan, no widths	17.17	12.48	5.75	6.80	7.83	12.25
GP, robust $\lambda=1$	24.95	9.24	4.52	5.32	6.08	10.31
GP, robust $\lambda=1$, no widths	17.45	12.05	5.54	6.50	7.42	12.08
GP, robust $\lambda=4$	28.27	8.61	4.21	5.01	5.73	9.78
G&T-MWKR	29.50	13.77	6.29	7.51	8.58	14.94
EST	42.30	13.37	5.66	6.79	7.83	14.64

before any realization is drawn, so no distributional assumption enters the problem formulation of Section 3, the interval-arithmetic fitness of Section 4, the evolution or the selection of any rule: the distribution-free character of the interval model is not affected. The uniform law is the least committal choice on a bounded support, and the measurement is repeated below under three other laws. To probe behavior under growing uncertainty we also widen every interval symmetrically about its midpoint by +20% and +40% (clipping negative bounds to zero), re-evaluating the same processing orders.

Under the makespan objective the width terminals make no difference to robustness either (Table 11): the featured rule and its ablated counterpart are statistically indistinguishable in $\bar{\varepsilon}$ (5.70 against 5.75 at the nominal

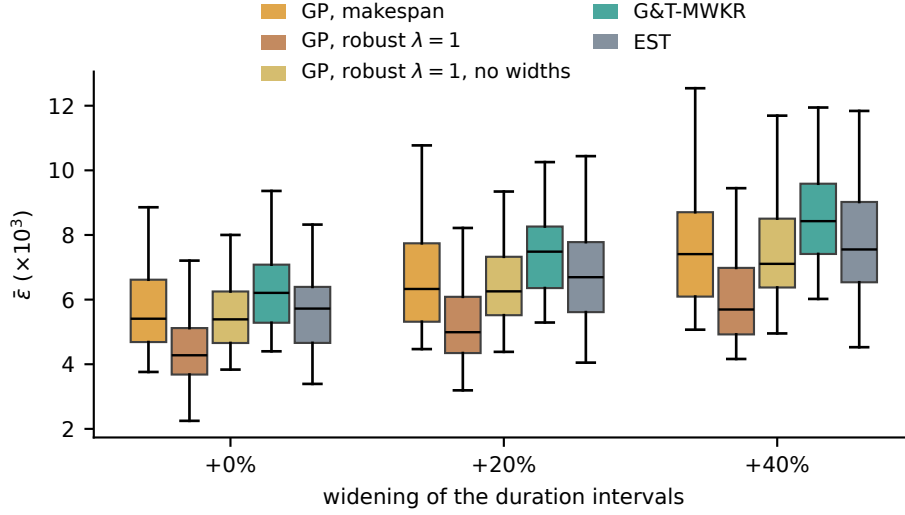


Figure 9: Per-instance $\bar{\varepsilon}$ by method, with the duration intervals at their nominal widths and widened by +20% and +40%. The robust $\lambda=1$ entries are the narrowest rule of that arm and its counterpart evolved without the width terminals, as in Table 11.

width; paired Wilcoxon over the 70 instances, $z = -0.17$). The rule is more robust than G&T-MWKR (6.29, $z = -4.42$, $p < 0.001$, $|r| = 0.61$) but not than EST (5.66, $z = 0.42$), although it improves on EST by 24.6 points of RE. A low deviation does not require a low makespan, so robustness must be optimized for directly.

The comparison with EST reverses once the normalization is removed. $\bar{\varepsilon}$ divides by $E[C_{\max}]$, and EST’s predicted makespan is 21% larger than the rule’s (2732 against 2259 time units on average), so the same absolute error reads as a smaller relative one. In absolute terms the rule deviates by 12.30 time units against EST’s 14.64, and the difference is significant ($z = -5.89$, $p < 0.001$, $|r| = 0.81$). The ordering against G&T-MWKR is unchanged and widens (12.30 against 14.94, $z = -6.83$, $|r| = 0.94$), and the makespan

arm and its ablation remain indistinguishable (12.30 against 12.25, $z = 0.61$, $p = 0.55$). The normalization therefore conceals an advantage of the rule rather than creating one.

The robust fitness achieves this. The narrowest rule of the $\lambda=1$ arm reduces $\bar{\varepsilon}$ from 5.70 to 4.52 ($z = -6.40$, $|r| = 0.88$ against the makespan rule), and the reduction requires the width terminals: under the same fitness, its ablated counterpart reaches 5.54 ($z = -6.25$, $|r| = 0.86$). At $\lambda=4$ the deviation falls to 4.21. The narrower predicted intervals of Section 7.5 thus correspond to smaller realized deviations, at the cost in expected makespan reported in the RE column: 24.95 and 28.27 against 17.71.

Part of that reduction is due to the denominator, and the absolute deviation quantifies it. The robust $\lambda=1$ arm carries an $E[\mathbf{C}_{\max}]$ that is 6.1% larger than the makespan arm's, so where $\bar{\varepsilon}$ falls by 20.7% the absolute deviation falls by 16.2%, from 12.30 to 10.31 time units; at $\lambda=4$ it reaches 9.78. Roughly a fifth of the apparent gain is therefore arithmetic and the rest is not: the paired contrasts are also significant on the absolute measure, by -1.99 time units against the makespan arm ($z = -5.55$, $p < 0.001$, $|r| = 0.76$) and by -1.78 against its own ablation ($z = -5.17$, $|r| = 0.71$).

Methods can also be compared at similar levels of RE, which separates the robustness gain from the expected makespan given up for it. Two entries of Table 11 allow that comparison without any adjustment. The robust $\lambda=4$ rule and G&T-MWKR predict almost the same makespan, 2454 against 2482 time units, which is 28.27 against 29.50 points of RE; the robust rule is marginally better a priori. Their realized deviations differ markedly: 9.78 against 14.94 time units, a reduction of 34.5%, smaller on 69 of the 70 instances ($z = -7.26$, $p < 0.001$, $|r| = 1.00$), and of the same sign and significance under the three other realization laws considered below. At matched

a-priori quality the robustness gain is therefore neither a denominator effect nor a makespan concession.

The comparisons above select one representative rule per arm; at the level of the method the ordering is the same. Recomputing $\bar{\varepsilon}$ at the nominal width for all 30 rules of each arm and pairing by seed, the makespan arm averages 5.67 ± 0.16 against 5.77 ± 0.14 for its ablation ($z = -2.58$, $p < 0.05$ after Holm’s correction over the three arm-level tests, $|r| = 0.54$), the robust $\lambda=1$ arm 5.50 ± 0.30 against 5.68 ± 0.11 for its own ($z = -3.26$, $p < 0.01$, $|r| = 0.68$), and the robust arm improves on the makespan arm with the same terminals ($z = -2.46$, $p < 0.05$, $|r| = 0.51$).

Two things follow. The width terminals do reduce $\bar{\varepsilon}$ slightly even under the makespan objective, a method-level effect too small to be detected in the single featured pair; and the arm-level reductions are modest, three percent or less, against the fifth separating the featured rules, so selecting the narrowest rule of the robust arm amplifies an effect that at the method level is consistent but small.

The estimate above draws each realization uniformly on its interval, which is one choice among several. Repeating the measurement with a symmetric triangular law, with a law skewed towards the upper endpoint, and with the single worst-case realization in which every duration takes its upper bound, leaves the ordering and every significant contrast intact. Averaged over the 70 instances, the robust $\lambda=1$ arm improves on the makespan arm by 1.22 time units under the triangular law, 12.16 under the skewed one and 29.01 in the worst case, all with $p < 0.001$; the makespan arm and its ablation remain indistinguishable under all three. The deviations themselves grow with the mass the law places near the upper endpoint, from 8.95 time units for the rule under the triangular law to 57.15 under the skewed one

and 138.35 in the worst case, but the comparison between methods does not depend on which law is used.

The measures above are averages over realizations, whereas a planner exposed to late completions is concerned with the worst ones. The conditional value-at-risk of Section 5.5, computed on the same $K=1000$ realizations, describes the 5% worst executions of each schedule and separates two questions. The first is how far those executions exceed the prediction. Under the uniform law the featured rule overruns its expected makespan by 33.46 time units on average over its worst 5% of executions, against 38.87 for G&T-MWKR and 40.81 for EST ($z = -5.04$ and $z = -5.61$, $p < 0.001$), and its ablated counterpart is indistinguishable (33.55, $z = -0.05$). The robust objective reduces the tail overrun in about the same proportion as the mean deviation: to 27.46 at $\lambda=1$, 17.9% less than the makespan rule ($z = -5.14$, $p < 0.001$, $|r| = 0.71$) and 5.68 less than its own ablation ($z = -5.32$, $|r| = 0.73$), and to 26.43 at $\lambda=4$. At matched a-priori quality, the robust $\lambda=4$ rule overruns by 26.43 against 38.87 for G&T-MWKR, 32.0% less ($z = -7.00$, $|r| = 0.96$). The second question is how long the worst executions are in absolute terms, and there the ordering follows expected makespan rather than robustness. Expressed as RE, the tail of the featured rule lies at 19.53% and those of the robust rules at 26.46% and 29.73%, higher on 68 and on 70 of the 70 instances, respectively. The robust objective therefore makes the prediction more reliable in the tail as well as on average, but it does not shorten the worst executions, whose length follows the expected makespan given up for that reliability. Against G&T-MWKR, whose expected makespan is close to its own, the $\lambda=4$ rule has a slightly shorter tail (29.73 against 31.59, $z = -2.41$, $p = 0.016$, $|r| = 0.33$). All of these contrasts keep their sign under the triangular and the skewed laws, un-

der which the tail overrun at $\lambda=1$ falls by 3.23 and 14.89 time units against the makespan rule, and all keep their significance except the shorter tail of the $\lambda=4$ rule against G&T-MWKR, which is marginal under the triangular law ($p = 0.044$).

Figure 9 shows the per-instance $\bar{\varepsilon}$ distributions. The ordering is stable as uncertainty grows, and the advantage of the robust rule widens with it: at +40% its deviation is 6.08 against 7.57 for the makespan rule and 8.58 for G&T-MWKR.

7.7. Sensitivity to the training set

The rules of this paper are evolved on four instances, TA11–TA14, fixed before any result was inspected. Whether the generalization figures rest on that particular quadruple is a question the design cannot answer by itself, so five further campaigns repeat the protocol unchanged — thirty independent evolutions each, same terminal set, same irace configuration of Section 5.4 — over different training sets: three alternative quadruples of the 20×15 class (TA15–TA18, TA17–TA20, and the even-numbered TA12, TA14, TA16, TA18), a pair (TA11–TA12) and an octuple (TA11–TA18). All six are evaluated on the 60 instances of the other size classes, which no campaign trains on, so the comparison surface is identical for every one of them, and the mean over the thirty rules is reported without any selection step, so the validation set plays no part in the comparison.

Table 12 gives the outcome. The six means span 0.66 points, from 18.95% to 19.61%, against a seed-to-seed standard deviation of 0.93–1.35 within each campaign: which instances are trained on matters less than which seed the evolution starts from. Paired by seed against the reference, no campaign separates from TA11–TA14 once Holm correction is applied over the five

Table 12: RE (%) over the 60 instances no campaign trains on, by training set: mean $\pm$ sd across 30 independently evolved rules, best rule, and the seed-paired Wilcoxon contrast against the reference.

Training set	$ T $	Mean $\pm$ sd	Best	Δ	p
TA11–TA14 (reference)	4	19.15 ± 1.35	17.89	—	—
TA15–TA18	4	19.46 ± 1.24	17.57	+0.31	0.229
TA17–TA20	4	19.53 ± 1.07	17.76	+0.37	0.025
TA12, 14, 16, 18	4	19.07 ± 0.93	17.49	−0.08	0.952
TA11–TA12	2	19.61 ± 1.23	17.53	+0.46	0.031
TA11–TA18	8	18.95 ± 1.03	17.38	−0.20	0.730

contrasts, the smallest adjusted p being 0.12. The size of the training set has an effect of the expected sign and small magnitude: halving the training set to two instances costs 0.46 points and doubling it to eight gains 0.20, both within the variation between seeds. The best rule of each campaign leads to the same conclusion, ranging from 17.38% to 17.89%, with the reference quadruple at the unfavorable extreme of that range.

What the training set does change is the shape of the rules. SLACK leads the terminal count everywhere, between 21% and 27%, but WKRW, the second most used terminal under the reference set, loses ground: it falls from 12.0% under TA11–TA14 to 5.8–11.8%, while EST rises from 9.7% to 10.8–14.4% under every other training set. Different instances select different tie-breakers around the same core, and reach the same performance with them.

7.8. Asymmetric intervals

Every instance used so far carries an interval symmetric about its crisp value, so what the width terminals contribute could belong to the generator

Table 13: The four arms re-evolved on asymmetric intervals and evaluated over the 70 asymmetric instances: mean $\pm$ sd across 15 seeds of RE, of the relative width of the predicted makespan interval, and of the absolute deviation of the executed makespan.

Arm	RE (%)	Width (%)	$ \Delta $
Makespan, full	25.08 ± 1.42	13.23 ± 0.72	13.36 ± 0.64
Makespan, no width	23.90 ± 1.18	13.08 ± 0.20	13.06 ± 0.26
Robust $\lambda=1$, full	25.93 ± 1.18	12.67 ± 0.20	12.75 ± 0.27
Robust $\lambda=1$, no width	23.89 ± 1.06	12.97 ± 0.18	13.03 ± 0.16

rather than to the problem. The four arms of Sections 7.4 and 7.5 were therefore re-evolved on the asymmetric counterparts of TA11–TA14 described in Section 5.1, with the same configuration and fifteen seeds per arm rather than thirty, a reduction imposed by the computational cost. Table 13 reports them over all 70 asymmetric instances, with the executed deviation estimated from 400 realizations of the fixed processing order. Levels of RE do not compare across the two benchmarks, since the right skew raises the expected makespan while the crisp lower bound does not move and every arm sits some six points higher; only the contrast between arms within a benchmark is meaningful.

The main result is reproduced. Under the robust objective the width terminals narrow the predicted interval from 12.97 to 12.67 ($z = -3.24$, $|r| = 0.95$) and the executed deviation from 13.03 to 12.75 ($z = -2.90$, $|r| = 0.85$), and the robust arm improves on the makespan arm carrying the same terminals by 0.55 points of width and 0.61 time units of deviation, the latter for every one of the fifteen seeds ($z = -3.41$, $|r| = 1.00$). These five contrasts survive Holm’s correction over the nine tests of the table, the largest adjusted p among them being 0.0101. The robustness that the width

terminals make attainable, and the objective under which they attain it, therefore do not depend on the symmetric generation scheme.

Under the makespan objective the results are weaker than on the symmetric benchmark. There, the width terminals made the predicted interval slightly but significantly narrower even though that objective does not reward it (Section 7.4); here they do not (13.23 against 13.08, $p = 0.89$), nor do they reduce the executed deviation (13.36 against 13.06, $p = 0.11$), and they cost 1.18 points of RE, a difference that does not survive the correction ($p = 0.10$ adjusted). The negative result for expected makespan holds on both benchmarks, and the remaining results are consistent with the boundary of Section 7.5: where the objective does not price width, the smaller terminal set is the better choice.

7.9. Limitations

Four limitations bound the scope of our conclusions. First, the evolved rules choose from the whole eligible set inside a semi-active decoder. Neither the conflict-set restriction that makes Giffler and Thompson’s procedure yield active schedules, used here only for the baselines, nor an insertion-based generation scheme was placed in the evolutionary loop, and both are known to help weaker constructors. Section 7.3 measures what the first of the two costs after the fact, and the rules lose accuracy: transplanted into the conflict set they were not evolved against, the arm gives up 4.2 points of RE. Whether evolving *inside* that decoder would recover them, or better, is open. Second, all rules were evolved on instances generated with the $\pm 15\%$ symmetric interval scheme standard in the IJSP literature; the robustness analysis of Section 7.6 does re-evaluate the resulting schedules under intervals widened by $+20\%$ and $+40\%$, and Section 7.8 re-evolves the four

arms on a right-skewed benchmark, where the robustness contribution of the width terminals is unchanged, but no rule was evolved under a different *width*, so how far these results carry to uncertainty of a different magnitude remains open. Third, absolute times are specific to our research-grade Python implementation; the rankings reported here, and the budget comparison of Section 6.5 measured in schedules constructed, do not depend on it. Finally, no learned policy is among the baselines. The one reinforcement-learning method published for the IJSP (Han et al., 2026) leaves part of the sequencing to be resolved during execution, so it does not belong to the single-pass class compared here, and it reports results on a different selection of classical instances.

8. Conclusions and Future Work

We presented what is, to the best of our knowledge, the first GP hyper-heuristic for job shop scheduling with interval durations: priority rules evolved over a terminal set that exposes both the worst-case values and the widths of the uncertain quantities, compact enough to be read and simplified by hand. The empirical conclusions are four. (i) Evolved rules outperform every deterministic constructive baseline on the 70-instance benchmark by a wide, statistically significant margin (18.99 ± 1.33 over 30 evolutions, 17.71% for the best rule, versus 29.5% for the strongest baseline, which the best rule outperforms on all 70 instances) at the cost of a single constructive pass, and the margin is not an artifact of the schedule generation scheme: holding the decoder fixed, either one, the evolved rules still outperform MWKR. (ii) They generalize zero-shot across all size classes, evolved on 20×15 and best on the unseen 50×15 ; since every terminal is a per-operation attribute

with no size-bound quantities, that transfer requires no retraining or rescaling. Nor does the result rest on the particular four instances trained on: across six training sets of two, four and eight instances the means span 0.66 points of RE, less than the spread between evolution seeds. (iii) Under the default expected-makespan objective, interval awareness at evolution time is unnecessary: removing the width terminals leaves it unchanged, and evolving on the crisp midpoint instances with scalar fitness does equally well. Under an objective that penalizes interval width, however, an objective with no crisp counterpart, the predicted intervals become substantially narrower and the deviation of the executed makespan from its prediction falls by about a fifth (4.52 against 5.70×10^{-3}); that reduction requires the width terminals, since under the same objective the ablated arm reaches only 5.54 and no value of λ narrows its intervals. The gain comes at the cost of a higher expected makespan, and the weight λ governs the trade-off. It extends to the tail of the distribution: over the worst 5% of executions the overrun beyond the prediction falls in about the same proportion, although those executions remain longer in absolute terms, as the expected makespan given up for the reliability determines. The robustness result survives a change of generator: re-evolved on a right-skewed benchmark, the width terminals again narrow both the predicted interval and the executed deviation under the robust objective, and again leave expected makespan unimproved under the default one.

(iv) Where an evolved rule stands against per-instance search is a question of budget, and it has a measurable answer. Our Python implementation of the genetic algorithm published for this problem, which shares decoder, evaluator and language with the rule, needs about 7.6×10^4 schedule constructions to match a single pass of the evolved rule, and at 1.3×10^5 it

has still not reached what the same rule attains with 1024 samples. Below roughly 10^5 constructions the evolved rule is the better use of the budget; above it, per-instance search becomes preferable, and the remaining distance to the best published ABC variants is not a matter of budget at all — a plain population method levels off well short of it — but of the neighborhood structures and local search those methods incorporate. What this paper contributes is therefore a high-quality, interpretable, fast-decision baseline for the IJSP, not a general solver; and its most useful role alongside a solver is to initialize it, since the same genetic algorithm seeded with the rule’s permutation is the only method that attains a lower error than the sampled rule.

Returning to the questions of Section 1: the evolution selects the classical scheduling attributes most often, with the width terminals present in most rules; whether the magnitude of the uncertainty is useful, and whether a robust objective yields more predictable executions, are answered by (iii); and where such rules stand against per-instance search, by (iv).

Future work follows four lines. First, optimizing the numeric weights of an evolved rule, which the sensitivity analysis of Section 7.1 suggests the evolution leaves unfinished. Two properties of this setting shape how such a search should be posed. Because operations are dispatched by *argmin*, the priority is invariant to adding a constant and to multiplying by a positive one, so the standard technique of linear scaling (Keijzer, 2003) cannot help and only the *relative* weights of the additive terms are free, $k - 1$ of them for a rule with k terms. And because the terminal set contains no ephemeral random constants, the evolution must build coefficients structurally, spending nodes of a bounded budget on arithmetic that a constant would express in one. Both an enriched terminal set and a local search over the relative

weights, whether applied after evolution or to the elites of each generation, follow from these observations. Second, richer symbolic models: ensembles of complementary rules (Gil-Gala et al., 2021) and co-evolution of construction and tie-breaking. Third, carrying the seeding result of Section 6.5 to the published metaheuristics themselves: here it is a genetic algorithm of our own that the rule improves, and whether the ABC variants, whose local search starts from a different kind of incumbent, gain as much is open. The pools of diverse sequences produced by GP-best-of- N are directly usable as seeded initial populations. Fourth, a controlled comparison with learned neural constructive policies under matched training and inference budgets, the head-to-head evaluation that the recent literature (Xu et al., 2025) identifies as missing.

Data availability

The GP hyper-heuristic is implemented from scratch in pure Python/NumPy, with no evolutionary-computation dependencies; evolved rules are stored as plain JSON expression trees and evaluated as drop-in dispatching strategies by the same environment used by every baseline. All benchmark instances, the evolved rules of every experimental arm, the primary result files behind each table and figure, and a self-contained implementation that re-derives the deposited results from scratch are openly available in a public repository. The link is withheld here because it would identify the author, and is provided to the editor on request and in the published version.

References

- Adams, J., Balas, E., Zawack, D., 1988. The shifting bottleneck procedure for job shop scheduling. *Management Science* 34, 391–401. doi:10.1287/mnsc.34.3.391.
- Afşar, S., Puente, J., Palacios, J.J., González-Rodríguez, I., Vela, C.R., 2025. Solving the energy-aware flexible job shop problem using a memetic algorithm. *Integrated Computer-Aided Engineering* 32, 397–414. doi:10.1177/10692509251353343.
- Allahverdi, A., 2022. A survey of scheduling problems with uncertain interval/bounded processing/setup times. *Journal of Project Management* 7, 255–264. doi:10.5267/j.jpm.2022.3.003.
- Allahverdi, A., Aydilek, H., Aydilek, A., 2014. Single machine scheduling problem with interval processing times to minimize mean weighted completion time. *Computers & Operations Research* 51, 200–207. doi:10.1016/j.cor.2014.06.003.
- Branke, J., Nguyen, S., Pickardt, C.W., Zhang, M., 2016. Automated design of production scheduling heuristics: a review. *IEEE Transactions on Evolutionary Computation* 20, 110–124. doi:10.1109/tevc.2015.2429314.
- Calamita, A., Meloni, C., Pranzo, M., Samà, M., 2025. Fast risk estimation for job shop scheduling solutions under interval uncertainty via machine learning. *Flexible Services and Manufacturing Journal* doi:10.1007/s10696-025-09640-7.
- Carlier, J., 1982. The one-machine sequencing problem. *European Journal of Operational Research* 11, 42–47. doi:10.1016/s0377-2217(82)80007-6.

- Díaz, H., González-Rodríguez, I., Palacios, J.J., Díaz, I., Vela, C.R., 2020. A genetic approach to the job shop scheduling problem with interval uncertainty, in: *Information Processing and Management of Uncertainty in Knowledge-Based Systems (IPMU)*, Springer. pp. 663–676. doi:10.1007/978-3-030-50143-3_52.
- Díaz, H., Palacios, J.J., Díaz, I., Vela, C.R., González-Rodríguez, I., 2022. Robust schedules for tardiness optimization in job shop with interval uncertainty. *Logic Journal of the IGPL* 31, 240–254. doi:10.1093/jigpal/jzac016.
- Díaz, H., Palacios, J.J., González-Rodríguez, I., Vela, C.R., 2023a. An elitist seasonal artificial bee colony algorithm for the interval job shop. *Integrated Computer-Aided Engineering* 30, 223–242. doi:10.3233/ica-230705.
- Díaz, H., Palacios, J.J., González-Rodríguez, I., Vela, C.R., 2023b. Fast elitist ABC for makespan optimisation in interval JSP. *Natural Computing* 22, 645–657. doi:10.1007/s11047-023-09953-2.
- Feo, T.A., Resende, M.G.C., 1995. Greedy randomized adaptive search procedures. *Journal of Global Optimization* 6, 109–133. doi:10.1007/BF01096763.
- Fisher, H., Thompson, G.L., 1963. Probabilistic learning combinations of local job-shop scheduling rules, in: Muth, J.F., Thompson, G.L. (Eds.), *Industrial Scheduling*. Prentice-Hall, Englewood Cliffs, NJ, pp. 225–251.
- Giffler, B., Thompson, G.L., 1960. Algorithms for solving production-

scheduling problems. *Operations Research* 8, 487–503. doi:10.1287/opre.8.4.487.

Gil-Gala, F.J., Mencía, C., Sierra, M.R., Varela, R., 2021. Learning ensembles of priority rules for online scheduling by hybrid evolutionary algorithms. *Integrated Computer-Aided Engineering* 28, 65–80. doi:10.3233/ica-200634.

Gil-Gala, F.J., Đurašević, M., Sierra, M.R., Puente, J., 2025a. Greedy randomised adaptive solution builder using priority rules, in: *Proceedings of the Genetic and Evolutionary Computation Conference Companion (GECCO '25)*, ACM. pp. 211–214. doi:10.1145/3712255.3726547.

Gil-Gala, F.J., Đurašević, M., Sierra, M.R., Varela, R., 2025b. Tree search hyper-heuristic with application to combinatorial optimization. *Journal of Heuristics* 31, 25. doi:10.1007/s10732-025-09560-7.

Han, P., Liu, Y., Huang, M., 2026. RAPO-RL-TAC: Risk-aware partial-order reinforcement learning with timed automata completion for the interval job shop problem. *Processes* 14, 2995. doi:10.3390/pr14182995.

Haupt, R., 1989. A survey of priority rule-based scheduling. *OR Spektrum* 11, 3–16. doi:10.1007/bf01721162.

Hildebrandt, T., Branke, J., 2015. On using surrogates with genetic programming. *Evolutionary Computation* 23, 343–367. doi:10.1162/evco_a_00133.

Ishibuchi, H., Tanaka, H., 1990. Multiobjective programming in optimization of the interval objective function. *European Journal of Operational Research* 48, 219–225. doi:10.1016/0377-2217(90)90375-L.

- Karunakaran, D., Mei, Y., Chen, G., Zhang, M., 2017. Dynamic job shop scheduling under uncertainty using genetic programming, in: *Intelligent and Evolutionary Systems*, Springer. pp. 195–210. doi:10.1007/978-3-319-49049-6_14.
- Kazimipour, B., Li, X., Qin, A.K., 2014. A review of population initialization techniques for evolutionary algorithms, in: *2014 IEEE Congress on Evolutionary Computation (CEC)*, IEEE. pp. 2585–2592. doi:10.1109/CEC.2014.6900618.
- Keijzer, M., 2003. Improving symbolic regression with interval arithmetic and linear scaling, in: *Genetic Programming (EuroGP)*, Springer. pp. 70–82. doi:10.1007/3-540-36599-0_7.
- Koza, J.R., 1992. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, Cambridge, MA.
- Lawrence, S., 1984. Resource constrained project scheduling: an experimental investigation of heuristic scheduling techniques (Supplement). Technical Report. Graduate School of Industrial Administration, Carnegie-Mellon University. Pittsburgh, PA.
- Lei, D., 2011. Population-based neighborhood search for job shop scheduling with interval processing time. *Computers & Industrial Engineering* 61, 1200–1208. doi:10.1016/j.cie.2011.07.010.
- Leon, V.J., Wu, S.D., Storer, R.H., 1994. Robustness measures and robust scheduling for job shops. *IIE Transactions* 26, 32–43. doi:10.1080/07408179408966626.

- López-Ibáñez, M., Dubois-Lacoste, J., Pérez Cáceres, L., Birattari, M., Stützle, T., 2016. The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives* 3, 43–58. doi:10.1016/j.orp.2016.09.002.
- Mei, Y., Nguyen, S., Xue, B., Zhang, M., 2017. An efficient feature selection algorithm for evolving job shop scheduling rules with genetic programming. *IEEE Transactions on Emerging Topics in Computational Intelligence* 1, 339–353. doi:10.1109/TETCI.2017.2743758.
- Moore, R.E., Kearfott, R.B., Cloud, M.J., 2009. *Introduction to Interval Analysis*. SIAM, Philadelphia, PA. doi:10.1137/1.9780898717716.
- Nguyen, S., Mei, Y., Zhang, M., 2017. Genetic programming for production scheduling: a survey with a unified framework. *Complex & Intelligent Systems* 3, 41–66. doi:10.1007/s40747-017-0036-x.
- Nguyen, S., Zhang, M., Johnston, M., Tan, K.C., 2014. Automatic design of scheduling policies for dynamic multi-objective job shop scheduling via cooperative coevolution genetic programming. *IEEE Transactions on Evolutionary Computation* 18, 193–208. doi:10.1109/TEVC.2013.2248159.
- Palacios, J.J., González-Rodríguez, I., Vela, C.R., Puente, J., 2015. Co-evolutionary makespan optimisation through different ranking methods for the fuzzy flexible job shop. *Fuzzy Sets and Systems* 278, 81–97. doi:10.1016/j.fss.2014.12.003.
- Panwalkar, S.S., Iskander, W., 1977. A survey of scheduling rules. *Operations Research* 25, 45–61. doi:10.1287/opre.25.1.45.

- Park, J., Chun, J., Kim, S.H., Kim, Y., Park, J., 2021. Learning to schedule job-shop problems: representation and policy learning using graph neural network and reinforcement learning. *International Journal of Production Research* 59, 3360–3377. doi:10.1080/00207543.2020.1870013.
- Poli, R., Langdon, W.B., McPhee, N.F., 2008. A Field Guide to Genetic Programming. Lulu Enterprises. Freely available at <http://www.gp-field-guide.org.uk>.
- Sotskov, Y.N., Lai, T.C., Werner, F., 2013. Measures of problem uncertainty for scheduling with interval processing times. *OR Spectrum* 35, 659–689. doi:10.1007/s00291-012-0306-3.
- Sotskov, Y.N., Matsveichuk, N.M., Hatura, V.D., 2020. Schedule execution for two-machine job-shop to minimize makespan with uncertain processing times. *Mathematics* 8, 1314. doi:10.3390/math8081314.
- Sprecher, A., Kolisch, R., Drexl, A., 1995. Semi-active, active, and non-delay schedules for the resource-constrained project scheduling problem. *European Journal of Operational Research* 80, 94–102. doi:10.1016/0377-2217(93)e0294-8.
- Taillard, E., 1993. Benchmarks for basic scheduling problems. *European Journal of Operational Research* 64, 278–285. doi:10.1016/0377-2217(93)90182-M.
- Đurašević, M., Jakobović, D., 2018. A survey of dispatching rules for the dynamic unrelated machines environment. *Expert Systems with Applications* 113, 555–569. doi:10.1016/j.eswa.2018.06.053.

- Xu, M., Mei, Y., Zhang, F., Zhang, M., 2025. Learn to optimise for job shop scheduling: a survey with comparison between genetic programming and reinforcement learning. *Artificial Intelligence Review* 58, 160. doi:10.1007/s10462-024-11059-9.
- Xu, W., Wu, W., Wang, Y., He, Y., Lei, Z., 2023. Flexible job-shop scheduling method based on interval grey processing time. *Applied Intelligence* 53, 14876–14891. doi:10.1007/s10489-022-04213-9.
- Zhang, C., Song, W., Cao, Z., Zhang, J., Tan, P.S., Xu, C., 2020. Learning to dispatch for job shop scheduling via deep reinforcement learning, in: *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 1621–1632.
- Zheng, P., Xiao, S., Zhang, P., Lv, Y., 2024. A two-individual-based evolutionary algorithm for flexible assembly job shop scheduling problem with uncertain interval processing times. *Applied Sciences* 14, 10304. doi:10.3390/app142210304.

Appendix A. Per-instance results

Table A.1 lists, for each of the 70 interval Taillard instances, the crisp lower bound and the single-pass RE (%) of the evolved rule and of the two strongest baselines: EST, the best of the plain dispatching rules, and G&T-MWKR, the best baseline overall.

Table A.1: Per-instance RE (%) of the evolved rule (GP) and of the two strongest baselines, EST and G&T-MWKR (single deterministic pass). LB is the crisp Taillard lower bound.

Inst.	LB	EST	G&T	GP	Inst.	LB	EST	G&T	GP
TA1	1231	49.9	26.1	19.3	TA36	1819	41.1	42.8	23.8
TA2	1244	24.9	27.8	14.2	TA37	1771	43.4	35.6	18.4
TA3	1218	33.0	33.2	14.1	TA38	1673	45.0	28.0	20.1
TA4	1175	32.0	27.7	13.0	TA39	1795	46.6	28.9	22.4
TA5	1224	27.2	33.0	14.7	TA40	1651	46.9	42.3	26.1
TA6	1238	26.8	18.2	13.7	TA41	1906	44.6	45.0	33.4
TA7	1227	30.7	26.1	17.7	TA42	1884	71.1	38.1	24.3
TA8	1217	23.7	25.0	14.8	TA43	1809	68.7	39.8	22.3
TA9	1274	37.7	28.2	20.0	TA44	1948	52.1	34.7	25.8
TA10	1241	37.0	22.1	15.4	TA45	1997	50.2	33.7	14.9
TA11	1357	56.2	30.7	17.5	TA46	1957	60.8	31.6	22.4
TA12	1367	48.8	34.6	17.6	TA47	1807	46.8	30.1	25.6
TA13	1342	60.2	26.9	13.3	TA48	1912	59.1	33.7	21.1
TA14	1345	39.5	29.7	13.2	TA49	1931	51.0	35.3	20.1
TA15	1339	48.1	30.6	16.7	TA50	1833	61.4	46.2	31.3
TA16	1360	45.2	34.0	15.6	TA51	2760	33.8	34.7	23.2
TA17	1462	58.9	17.7	13.9	TA52	2756	38.0	26.5	12.7
TA18	1377	43.8	31.8	23.6	TA53	2717	34.1	18.3	11.8
TA19	1332	45.8	29.3	20.0	TA54	2839	20.9	16.1	8.2
TA20	1348	45.7	24.1	15.1	TA55	2679	31.7	29.2	15.5
TA21	1642	33.4	28.0	20.2	TA56	2781	34.8	19.8	10.9
TA22	1561	44.2	30.8	17.4	TA57	2943	32.9	20.0	9.6
TA23	1518	42.8	35.3	17.8	TA58	2885	37.4	26.3	13.6
TA24	1644	56.3	23.8	14.1	TA59	2655	26.6	28.2	16.7
TA25	1558	45.8	33.8	16.6	TA60	2723	41.7	19.8	15.5
TA26	1591	42.8	31.2	26.9	TA61	2868	45.7	25.4	15.4
TA27	1652	50.0	31.1	21.4	TA62	2869	43.0	27.6	17.2
TA28	1603	36.8	26.8	12.4	TA63	2755	38.8	23.8	13.4
TA29	1583	28.8	32.0	17.2	TA64	2702	38.2	28.4	12.2
TA30	1528	34.2	36.1	17.0	TA65	2725	42.9	30.0	20.5
TA31	1764	30.7	34.4	24.5	TA66	2845	41.8	28.0	11.9
TA32	1774	42.7	33.3	19.0	TA67	2825	31.7	26.1	15.2
TA33	1788	51.3	34.8	27.5	TA68	2784	50.1	19.1	10.6
TA34	1828	45.9	31.8	20.7	TA69	3071	35.3	22.8	12.9
TA35	2007	30.3	24.1	8.4	TA70	2995	42.9	23.4	16.7